

Chapter 3. Processes and jobs

In this chapter, we'll explain the data structures and algorithms that deal with processes and jobs in Windows. First we'll take a general look at process creation. Then we'll examine the internal structures that make up a process. Next we'll look at protected processes and how they differ from non-protected ones. After that we outline the steps involved in creating a process (and its initial thread). The chapter concludes with a description of jobs.

Because processes touch so many components in Windows, a number of terms and data structures (such as working sets, threads, objects and handles, system memory heaps, and so on) are referred to in this chapter but are explained in detail elsewhere in the book. To fully understand this chapter, you need to be familiar with the terms and concepts explained in [Chapter 1, “Concepts and tools,”](#) and [Chapter 2, “System architecture,”](#) such as the difference between a process and a thread, the Windows virtual address space layout, and the difference between user mode and kernel mode.

Creating a process

The Windows API provides several functions for creating processes. The simplest is `CreateProcess`, which attempts to create a process with the same access token as the creating process. If a different token is required, `CreateProcessAsUser` can be used, which accepts an extra argument (the first)—a handle to a token object that was already somehow obtained (for example, by calling the `LogonUser` function).

Other process creation functions include `CreateProcessWithTokenW` and `CreateProcessWithLogonW` (both part of `advapi32.dll`). `CreateProcessWithTokenW` is similar to `CreateProcessAsUser`, but the

two differ in the privileges required for the caller. (Check the Windows SDK documentation for the specifics.) `CreateProcessWithLogonW` is a handy shortcut to log on with a given user's credentials and create a process with the obtained token in one stroke. Both call the Secondary Logon service (`seclogon.dll`, hosted in a `SvcHost.Exe`) by making a Remote Procedure Call (RPC) to do the actual process creation.

`SecLogon` executes the call in its internal `SlrCreateProcessWithLogon` function, and if all goes well, eventually calls `CreateProcessAsUser`.

The `SecLogon` service is configured by default to start manually, so the first time `CreateProcessWithTokenW` or `CreateProcessWithLogonW` is called, the service is started. If the service fails to start (for example, an administrator can configure the service to be disabled), these functions will fail. The `runas` command-line utility, which you may be familiar with, makes use of these functions.

Figure 3-1 shows the call graph described above.

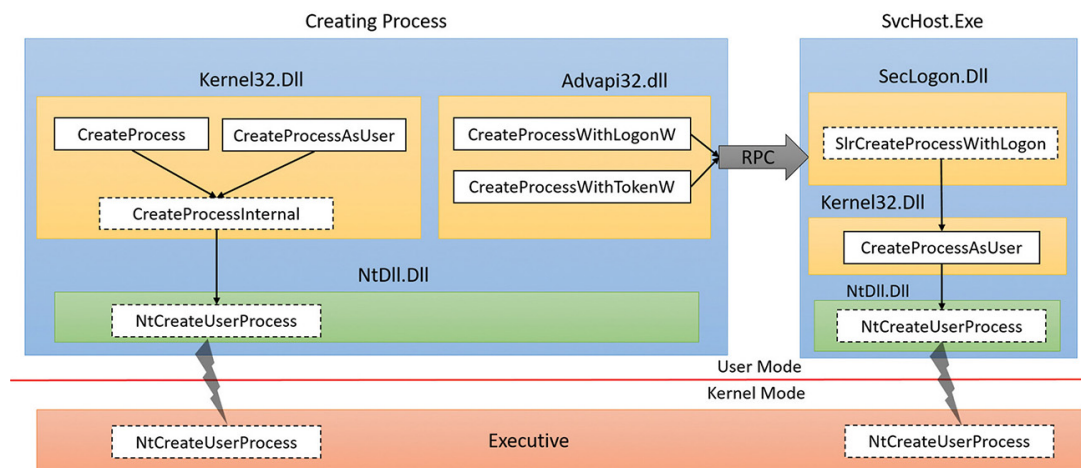


FIGURE 3-1 Process creation functions. Functions marked with dotted boxes are internal.

All the above documented functions expect a proper Portable Executable (PE) file (although the EXE extension is not strictly required), batch file, or 16-bit COM application. Beyond that, they have no knowledge of how to connect files with certain extensions (for example, `.txt`) to an executable (for example, Notepad). This is something that is provided by the Windows Shell, in functions such as `ShellExecute` and `ShellExecuteEx`. These functions can accept any file (not just executables) and try to locate the executable to run based on the file exten-

sions and the registry settings at `HKEY_CLASSES_ROOT`. (See Chapter 9, “Management mechanisms,” in *Windows Internals Part 2* for more on this.) Eventually, `ShellExecute ( Ex )` calls `CreateProcess` with a proper executable and appends appropriate arguments on the command line to achieve the user’s intention (such as editing a TXT file by appending the file name to `Notepad.exe`).

Ultimately, all these execution paths lead to a common internal function, `CreateProcessInternal`, which starts the actual work of creating a user-mode Windows process. Eventually (if all goes well), `CreateProcessInternal` calls `NtCreateUserProcess` in `Ntdll.dll` to make the transition to kernel mode and continue the kernel-mode part of process creation in the function with the same name (`NtCreateUserProcess`), part of the Executive.

CreateProcess* functions arguments

It’s worthwhile to discuss the arguments to the `CreateProcess *` family of functions, some of which will be referred to in the section on the flow of `CreateProcess`. A process created from user mode is always created with one thread within it. This is the thread that eventually will execute the main function of the executable. Here are the important arguments to the `CreateProcess*` functions:

- For `CreateProcessAsUser` and `CreateProcessWithTokenW`, the token handle under which the new process should execute. Similarly, for `CreateProcessWithLogonW`, the username, domain and password are required.
- The executable path and command-line arguments.
- Optional security attributes applied to the new process and thread object that’s about to be created.
- A Boolean flag indicating whether all handles in the current (creating) process that are marked inheritable should be inherited (copied) to the

new process. (See Chapter 8, “System mechanisms,” in Part 2 for more on handles and handle inheritance.)

■ Various flags that affect process creation. Here are some examples. (Check the Windows SDK documentation for a complete list.)

- **CREATE_SUSPENDED** This creates the initial thread of the new process in the suspended state. A later call to `ResumeThread` will cause the thread to begin execution.
- **DEBUG_PROCESS** The creating process is declaring itself to be a debugger, creating the new process under its control.
- **EXTENDED_STARTUPINFO_PRESENT** The extended `STARTUPINFOEX` structure is provided instead of `STARTUPINFO` (described below).

■ An optional environment block for the new process (specifying environment variables). If not specified, it will be inherited from the creating process.

■ An optional current directory for the new process. (If not specified, it uses the one from the creating process.) The created process can later call `SetCurrentDirectory` to set a different one. The current directory of a process is used in various non-full path searches (such as when loading a DLL with a filename only).

■ A `STARTUPINFO` or `STARTUPINFOEX` structure that provides more configuration for process creation. `STARTUPINFOEX` contains an additional opaque field that represents a set of process and thread attributes that are essentially an array of key/value pairs. These attributes are filled by calling `UpdateProcThreadAttributes` once for each attribute that's needed. Some of these attributes are undocumented and used internally, such as when creating store apps, as described in the next section.

■ A `PROCESS_INFORMATION` structure that is the output of a successful process creation. This structure holds the new unique process ID, the new unique thread ID, a handle to the new process and a handle to the

new thread. The handles are useful for the creating process if it wants to somehow manipulate the new process or thread in some way after creation.

Creating Windows modern processes

Chapter 1 described the new types of applications available starting from Windows 8 and Windows Server 2012. The names of these apps have changed over time, but we'll refer to them as modern apps, UWP apps, or immersive processes, to distinguish them from the classic, also known as desktop, applications.

Creating a modern application process requires more than just calling `CreateProcess` with the correct executable path. There are some required command-line arguments. Yet another requirement is adding an undocumented process attribute (using `UpdateProcThreadAttribute`) with a key named `PROC_THREAD_ATTRIBUTE_PACKAGE_FULL_NAME` with the value set to the full store app package name. Although this attribute is undocumented, there are other ways (from an API perspective) to execute a store app. For example, the Windows API includes a COM interface called `IApplicationActivationManager` that is implemented by a COM class with a CLSID named `CLSID_ApplicationActivationManager`. One of the methods in the interface is `ActivateApplication`, which can be used to launch a store app after obtaining something known as *AppUserModelId* from the store app full package name by calling `GetPackageApplicationIds`. (See the Windows SDK for more information on these APIs.)

Package names and the way a store app is typically created, from a user tapping on a modern app tile, eventually leading to `CreateProcess`, is discussed in Chapter 9 in Part 2.

Creating other kinds of processes

Although Windows applications launch either classic or modern applications, the Executive includes support for additional kinds of processes that must be started by bypassing the Windows API, such as native processes, minimal processes, or Pico processes. For example, we described in [Chapter 2](#) the existence of Smss, the Session Manager, which is an example of a native image. Since it is created directly by the kernel, it obviously does not use the `CreateProcess` API, but instead calls directly into `NtCreateUserProcess`. Similarly, when Smss creates Autochk (the check disk utility) or Csrss (the Windows subsystem process), the Windows API is also not available, and `NtCreateUserProcess` must be used. Additionally, native processes cannot be created from Windows applications, as the `CreateProcessInternal` function will reject images with the native subsystem image type. To alleviate these complications, the native library, Ntdll.dll, includes an exported helper function called `RtlCreateUserProcess`, providing a simpler wrapper around `NtCreateUserProcess`.

As its name suggests, `NtCreateUserProcess` is used for the creation of user-mode processes. However, as we saw in [Chapter 2](#), Windows also includes a number of kernel-mode processes, such as the System process and the Memory Compression processes (which are minimal processes), plus the possibility of Pico processes managed by a provider such as the Windows Subsystem for Linux. The creation of such processes is instead provided by the `NtCreateProcessEx` system call, with certain capabilities reserved solely for kernel-mode callers (such as the creation of minimal processes).

Finally, Pico providers call a helper function, which takes care of both creating the minimal process as well as initializing its Pico provider context—`PspCreatePicoProcess`. This function is not exported, and is only available to Pico providers through their special interface.

As we'll see in the flow section later in this chapter, although `NtCreateProcessEx` and `NtCreate-UserProcess` are different system

calls, the same internal routines are used to perform the work:

`PspAllocateProcess` and `PspInsertProcess`. All the possible ways we've enumerated so far to create a process, and any ways you can imagine, from a WMI PowerShell cmdlet to a kernel driver, will end up there.

Process internals

This section describes the key Windows process data structures maintained by various parts of the system and describes different ways and tools to examine this data.

Each Windows process is represented by an executive process (`EPROCESS`) structure. Besides containing many attributes relating to a process, an `EPROCESS` contains and points to a number of other related data structures. For example, each process has one or more threads, each represented by an executive thread (`ETHREAD`) structure. (Thread data structures are explained in [Chapter 4, “Threads”](#).)

The `EPROCESS` and most of its related data structures exist in system address space. One exception is the Process Environment Block (PEB), which exists in the process (user) address space (because it contains information accessed by user-mode code). Additionally, some of the process data structures used in memory management, such as the working set list, are valid only within the context of the current process, because they are stored in process-specific system space. (See [Chapter 5, “Memory management,”](#) for more information on process address space.)

For each process that is executing a Windows program, the Windows subsystem process (Csrss) maintains a parallel structure called the `CSR_PROCESS`. Additionally, the kernel-mode part of the Windows subsystem (Win32k.sys) maintains a per-process data structure, `W32PROCESS`, which is created the first time a thread calls a Windows `USER` or `GDI` function that is implemented in kernel mode. This happens as soon as the User32.dll library is loaded. Typical functions that

cause this library to be loaded are `CreateWindow ( Ex )` and `GetMessage` .

Since the kernel-mode Windows subsystem makes heavy use of DirectX-based hardware accelerated graphics, the Graphics Device Interface (GDI) component infrastructure causes the DirectX Graphics Kernel (Dxgkrnl.sys) to initialize a structure of its own, `DXGPROCESS` . This structure contains information for DirectX objects (surfaces, shaders, etc.) and the GPGPU-related counters and policy settings for both computational and memory management-related scheduling.

Except for the `idle` process, every `EPROCESS` structure is encapsulated as a process object by the executive object manager (described in Chapter 8 in Part 2). Because processes are not named objects, they are not visible in the WinObj tool (from Sysinternals). You can, however, see the Type object called *Process* in the \ObjectTypes directory (in WinObj). A handle to a process provides, through use of the process-related APIs, access to some of the data in the `EPROCESS` structure and in some of its associated structures.

Many other drivers and system components, by registering process-creation notifications, can choose to create their own data structures to track information they store on a per-process basis. (The executive functions `PsSetCreateProcessNotifyRoutine(Ex, Ex2)` allow this and are documented in the WDK.) When one discusses the overhead of a process, the size of such data structures must often be taken into consideration, although it is nearly impossible to obtain an accurate number. Additionally, some of these functions allow such components to disallow, or block, the creation of processes. This provides anti-malware vendors with an architectural way to add security enhancements to the operating system, either through hash-based blacklisting or other techniques.

First let's focus on the Process object. [Figure 3-2](#) shows the key fields in an `EPROCESS` structure.

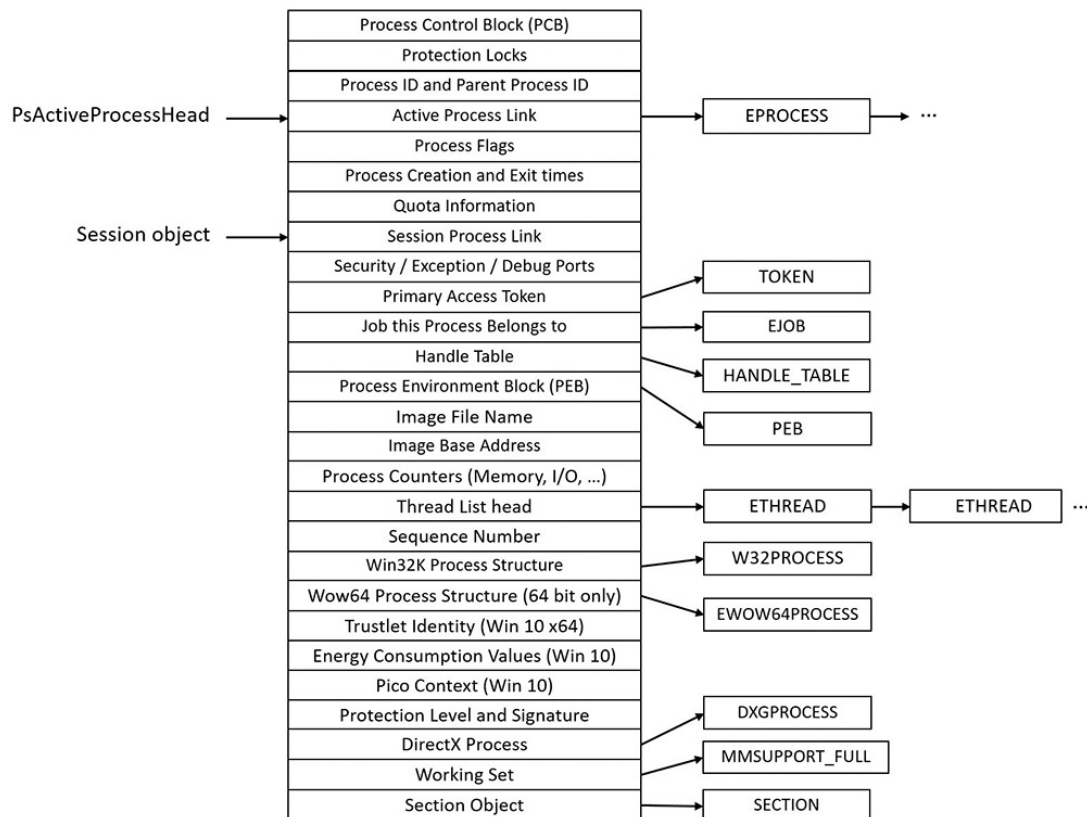


FIGURE 3-2 Important fields of the executive process structure.

Similar to the way the kernel's APIs and components are divided into isolated and layered modules with their own naming conventions, the data structures for a process follow a similar design. As shown in **Figure 3-2**, the first member of the executive process structure is called *Pcb* (*Process Control Block*). It is a structure of type `KPROCESS`, for *kernel process*. Although routines in the executive store information in the `EPROCESS`, the dispatcher, scheduler, and interrupt/time accounting code—being part of the operating system kernel—use the `KPROCESS` instead. This allows a layer of abstraction to exist between the executive's high-level functionality and its underlying low-level implementation of certain functions, and helps prevent unwanted dependencies between the layers. **Figure 3-3** shows the key fields in a `KPROCESS` structure.

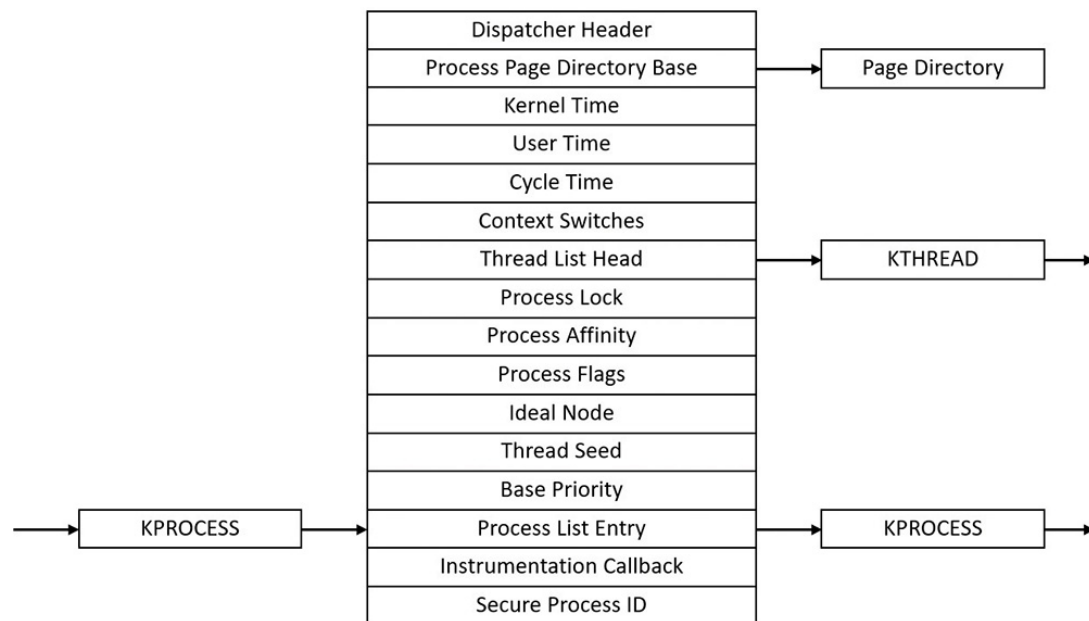


FIGURE 3-3 Important fields of the kernel process structure.

EXPERIMENT: DISPLAYING THE FORMAT OF AN EPROCESS STRUCTURE

For a list of the fields that make up an `EPROCESS` structure and their offsets in hexadecimal, type `dt nt!_eprocess` in the kernel debugger. (See [Chapter 1](#) for more information on the kernel debugger and how to perform kernel debugging on the local system.) The output (truncated for the sake of space) on a 64-bit Windows 10 system looks like this:

[Click here to view code image](#)

```
lkd> dt nt!_eprocess
+0x000 Pcb          : _KPROCESS
+0x2d8 ProcessLock  : _EX_PUSH_LOCK
+0x2e0 RundownProtect : _EX_RUNDOWN_REF
+0x2e8 UniqueProcessId : Ptr64 Void
+0x2f0 ActiveProcessLinks : _LIST_ENTRY
...
+0x3a8 Win32Process  : Ptr64 Void
+0x3b0 Job           : Ptr64 _EJOB
...
+0x418 ObjectTable   : Ptr64 _HANDLE_TABLE
+0x420 DebugPort     : Ptr64 Void
+0x428 WoW64Process  : Ptr64 _EWOW64PROCESS
...
+0x758 SharedCommitCharge : Uint8B
+0x760 SharedCommitLock : _EX_PUSH_LOCK
+0x768 SharedCommitLinks : _LIST_ENTRY
+0x778 AllowedCpuSets   : Uint8B
+0x780 DefaultCpuSets   : Uint8B
+0x778 AllowedCpuSetsIndirect : Ptr64 Uint8B
+0x780 DefaultCpuSetsIndirect : Ptr64 Uint8B
```

The first member of this structure (`Pcb`) is an embedded structure of type `KPROCESS` . This is where scheduling and time-accounting data is stored. You can display the format of the kernel process structure in the same way as the `EPROCESS` :

[Click here to view code image](#)

```
lkd> dt nt!_kprocess
+0x000 Header          : _DISPATCHER_HEADER
+0x018 ProfileListHead : _LIST_ENTRY
+0x028 DirectoryTableBase : Uint8B
+0x030 ThreadListHead  : _LIST_ENTRY
+0x040 ProcessLock      : Uint4B
...
+0x26c KernelTime       : Uint4B
+0x270 UserTime         : Uint4B
+0x274 LdtFreeSelectorHint : Uint2B
+0x276 LdtTableLength   : Uint2B
+0x278 LdtSystemDescriptor : _KGDTENTRY64
+0x288 LdtBaseAddress   : Ptr64 Void
+0x290 LdtProcessLock   : _FAST_MUTEX
+0x2c8 InstrumentationCallback : Ptr64 Void
+0x2d0 SecurePid        : Uint8B
```

The `dt` command also enables you to view the specific contents of one field or multiple fields by typing their names following the structure name. For example, typing **`dt nt!_eprocess UniqueProcessId`** displays the process ID field. In the case of a field that represents a structure—such as the `Pcb` field of `EPROCESS`, which contains the `KPROCESS` substructure—adding a period after the field name will cause the debugger to display the substructure. For example, an alternative way to see the `KPROCESS` is to type **`dt nt!_eprocess Pcb.`** You can continue to recurse this way by adding more field names (within `KPROCESS`) and so on. Finally, the `-r` switch of the `dt` command allows you to recurse through all the substructures. Adding a number after the switch controls the depth of recursion the command will follow.

The `dt` command used as shown earlier shows the format of the selected structure, not the contents of any particular instance of that structure type. To show an instance of an actual process, you can specify the address of an `EPROCESS` structure as an argument to the `dt` command. You can get the addresses of almost all of the `EPROCESS`

structures in the system by using the `!process 0 0` command (the exception being the system idle process). Because the `KPROCESS` is the first thing in the `EPROCESS`, the address of an `EPROCESS` will also work as the address of a `KPROCESS` with `dt _kprocess`.

EXPERIMENT: USING THE KERNEL DEBUGGER !PROCESS COMMAND

The kernel debugger `!process` command displays a subset of the information in a process object and its associated structures. This output is arranged in two parts for each process. First you see the information about the process, as shown here. When you don't specify a process address or ID, `!process` lists information for the process owning the thread currently running on CPU 0, which will be WinDbg itself (or `livekd` if it's used in lieu of WinDbg) on a single-processor system.

[Click here to view code image](#)

```
lkd> !process
PROCESS fffffe0011c3243c0
  SessionId: 2 Cid: 0e38 Peb: 5f2f1de000 ParentCid: 0f08
  DirBase: 38b3e000 ObjectTable: fffffc000a2b22200 HandleCount:
<Data Not Accessible>
  Image: windbg.exe
  VadRoot fffffe0011badae60 Vads 117 Clone 0 Private 3563. Modified
228. Locked 1.
  DeviceMap fffffc000984e4330
  Token                fffffc000a13f39a0
  ElapsedTime           00:00:20.772
  UserTime              00:00:00.000
  KernelTime           00:00:00.015
  QuotaPoolUsage[PagedPool] 299512
  QuotaPoolUsage[NonPagedPool] 16240
  Working Set Sizes (now,min,max) (9719, 50, 345) (38876KB, 200KB,
1380KB)
  PeakWorkingSetSize      9947
  VirtualSize            2097319 Mb
```

PeakVirtualSize	2097321 Mb
PageFaultCount	13603
MemoryPriority	FOREGROUND
BasePriority	8
CommitCharge	3994
Job	ffffe0011b853690

After the basic process output comes a list of the threads in the process. That output is explained in the “[Experiment: Using the kernel debugger !thread command](#)” section in [Chapter 4](#).

Other commands that display process information include `!handle`, which dumps the process handle table (described in more detail in the section “Object handles and the process handle table” in Chapter 8 in Part 2). Process and thread security structures are described in [Chapter 7](#), “[Security](#).”

Note that the output gives the address of the PEB. You can use this with the `!peb` command shown in the next experiment to see a friendly view of the PEB of an arbitrary process or you can use the regular `dt` command with the `_PEB` structure. However, because the PEB is in the user-mode address space, it is valid only within the context of its own process. To look at the PEB of another process, you must first switch WinDbg to that process. You can do this with the `.process /P` command, followed by the `EPROCESS` pointer.

If you’re using the latest Windows 10 SDK, the updated version of WinDbg will include an intuitive hyperlink under the PEB address, which you can click to automatically execute both the `.process` command and the `!peb` command.

The PEB lives in the user-mode address space of the process it describes. It contains information needed by the image loader, the heap manager, and other Windows components that need to access it from user mode; it would be too expensive to expose all that information

through system calls. The `EPROCESS` and `KPROCESS` structures are accessible only from kernel mode. The important fields of the PEB are illustrated in **Figure 3-4** and are explained in more detail later in this chapter.

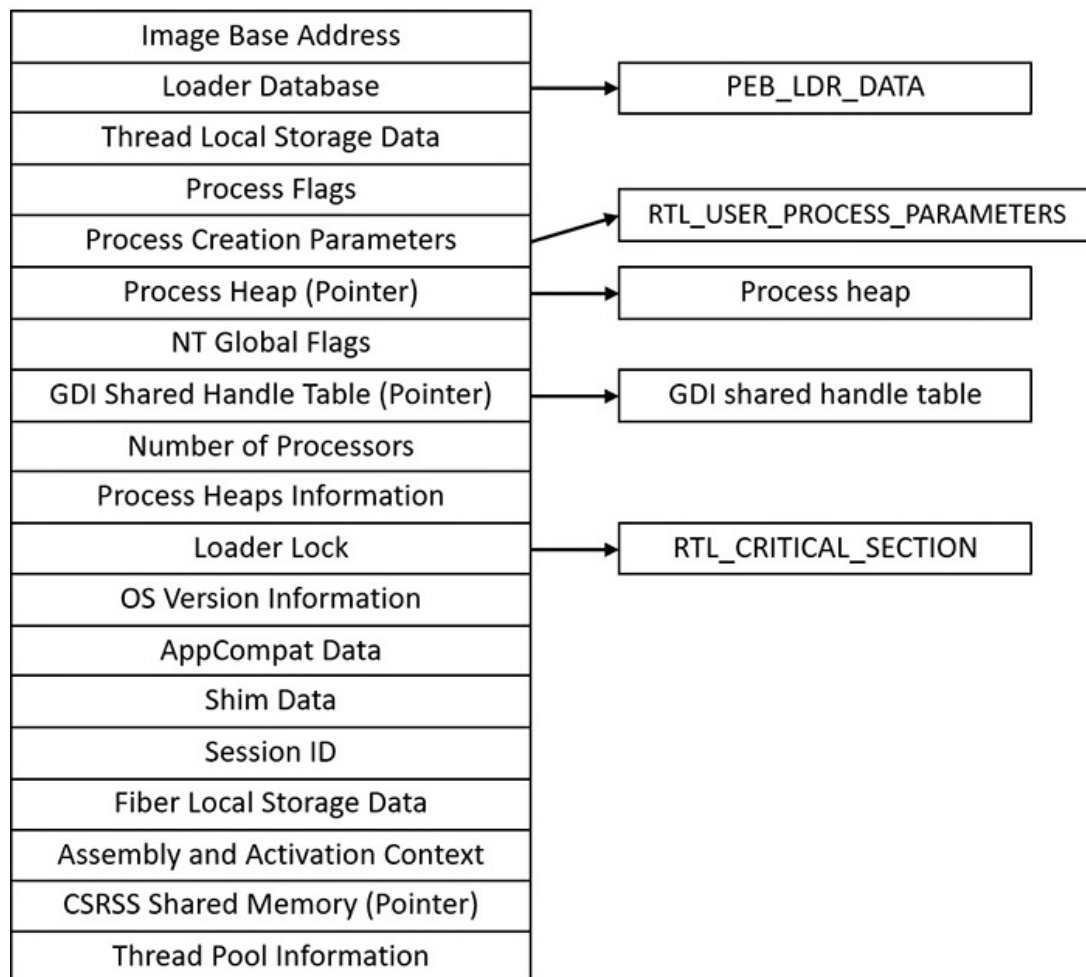


FIGURE 3-4 Important fields of the Process Environment Block.

EXPERIMENT: EXAMINING THE PEB

You can dump the PEB structure with the `!peb` command in the kernel debugger, which displays the PEB of the process that owns the currently running thread on CPU 0. By using the information in the previous experiment, you can also use the PEB pointer as an argument to the command.

[Click here to view code image](#)

```
lkd> .process /P fffff0011c3243c0 ; !peb 5f2f1de000
PEB at 0000003561545000
  InheritedAddressSpace: No
  ReadImageFileExecOptions: No
  BeingDebugged: No
  ImageBaseAddress: 00007ff64fa70000
  Ldr 00007ffdf52f5200
  Ldr.Initialized: Yes
  Ldr.InInitializationOrderModuleList: 000001d3d22b3630 .
000001d3d6cddb60
  Ldr.InLoadOrderModuleList: 000001d3d22b3790 .
000001d3d6cddb40
  Ldr.InMemoryOrderModuleList: 000001d3d22b37a0 .
000001d3d6cddb50
      Base TimeStamp      Module
      7ff64fa70000 56ccafdd Feb 23 21:15:41 2016
C:\dbg\x64\windbg.exe
      7ffdf51b0000 56cbf9dd Feb 23 08:19:09 2016
C:\WINDOWS\SYSTEM32\ntdll.dll
      7ffdf2c10000 5632d5aa Oct 30 04:27:54 2015
C:\WINDOWS\system32\KERNEL32.DLL
  ...
```

The `CSR_PROCESS` structure contains information about processes that is specific to the Windows subsystem (Csrss). As such, only Windows applications have a `CSR_PROCESS` structure associated with them (for

example, Smss does not). Additionally, because each session has its own instance of the Windows subsystem, the `CSR_PROCESS` structures are maintained by the Csrss process within each individual session. The basic structure of the `CSR_PROCESS` is illustrated in **Figure 3-5** and is explained in more detail later in this chapter.

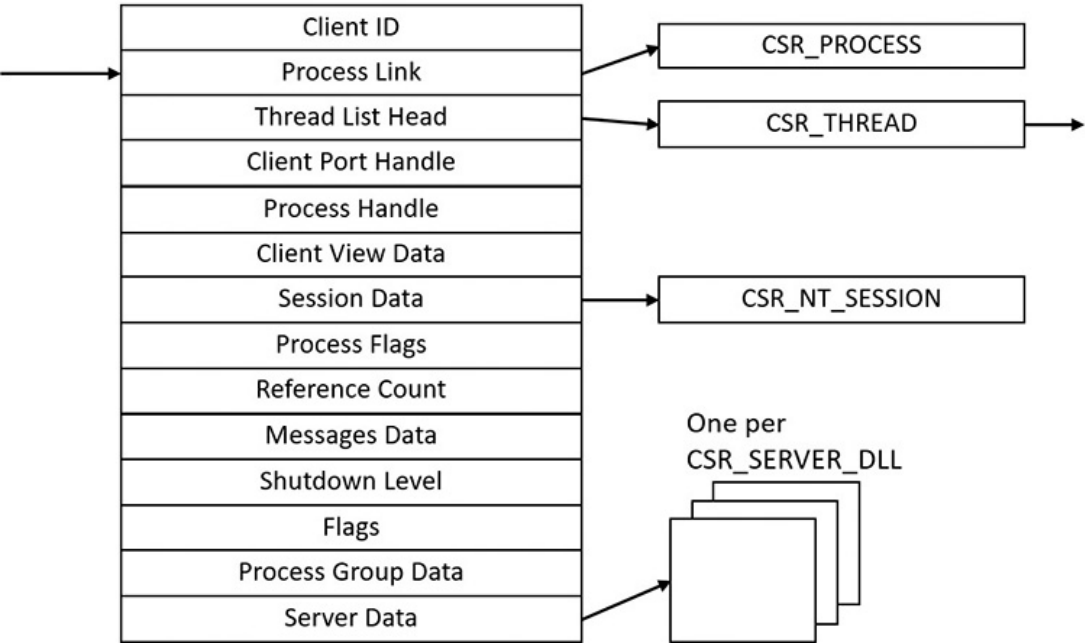


FIGURE 3-5 Fields of the CSR process structure.

EXPERIMENT: EXAMINING THE CSR_PROCESS

Csrss processes are protected (see later in this chapter for more on protected processes), so it's not possible to attach a user mode debugger to a Csrss process (not even with elevated privileges or non-invasively). Instead, we'll use the kernel debugger.

First, list the existing Csrss processes:

[Click here to view code image](#)

```
lkd> !process 0 0 csrss.exe
PROCESS fffff00077ddf080
    SessionId: 0 Cid: 02c0 Peb: c4e3fc0000 ParentCid: 026c
    DirBase: ObjectTable: fffff0004d15d040 HandleCount: 543.
    Image: csrss.exe

PROCESS fffff00078796080
    SessionId: 1 Cid: 0338 Peb: d4b4db4000 ParentCid: 0330
    DirBase: ObjectTable: fffff0004ddff040 HandleCount: 514.
    Image: csrss.exe
```

Next, take any one of them and change the debugger context to point to the particular process so that its user mode modules are visible:

[Click here to view code image](#)

```
lkd> .process /r /P fffff00078796080
Implicit process is now fffff000'78796080
Loading User Symbols
.....
```

The `/p` switch changes the process context of the debugger to the provided process object (`EPROCESS` , mostly needed in live debugging) and `/r` requests loading of user mode symbols. Now you can look at the modules themselves using the `lm` command or look at the `CSR_PROCESS` structure:

[Click here to view code image](#)

```
lkd> dt csrss!_csr_process
+0x000 ClientId      : _CLIENT_ID
+0x010 ListLink      : _LIST_ENTRY
+0x020 ThreadList    : _LIST_ENTRY
+0x030 NtSession     : Ptr64 _CSR_NT_SESSION
+0x038 ClientPort    : Ptr64 Void
+0x040 ClientViewBase : Ptr64 Char
+0x048 ClientViewBounds : Ptr64 Char
+0x050 ProcessHandle  : Ptr64 Void
+0x058 SequenceNumber : Uint4B
+0x05c Flags         : Uint4B
+0x060 DebugFlags     : Uint4B
+0x064 ReferenceCount : Int4B
+0x068 ProcessGroupId : Uint4B
+0x06c ProcessGroupSequence : Uint4B
+0x070 LastMessageSequence : Uint4B
+0x074 NumOutstandingMessages : Uint4B
+0x078 ShutdownLevel : Uint4B
+0x07c ShutdownFlags : Uint4B
+0x080 Luid          : _LUID
+0x088 ServerDllPerProcessData : [1] Ptr64 Void
```

The `W32PROCESS` structure is the final system data structure associated with processes that we'll look at. It contains all the information that the Windows graphics and window management code in the kernel (Win32k) needs to maintain state information about GUI processes (which were defined earlier as processes that have done at least one `USER` / `GDI` system call). The basic structure of the `W32PROCESS` is illustrated in **Figure 3-6**. Unfortunately, since type information for Win32k structures is not available in public symbols, we can't easily show you an experiment displaying this information. Either way, discussion of graphics-related data structures and concepts is beyond the scope of this book.

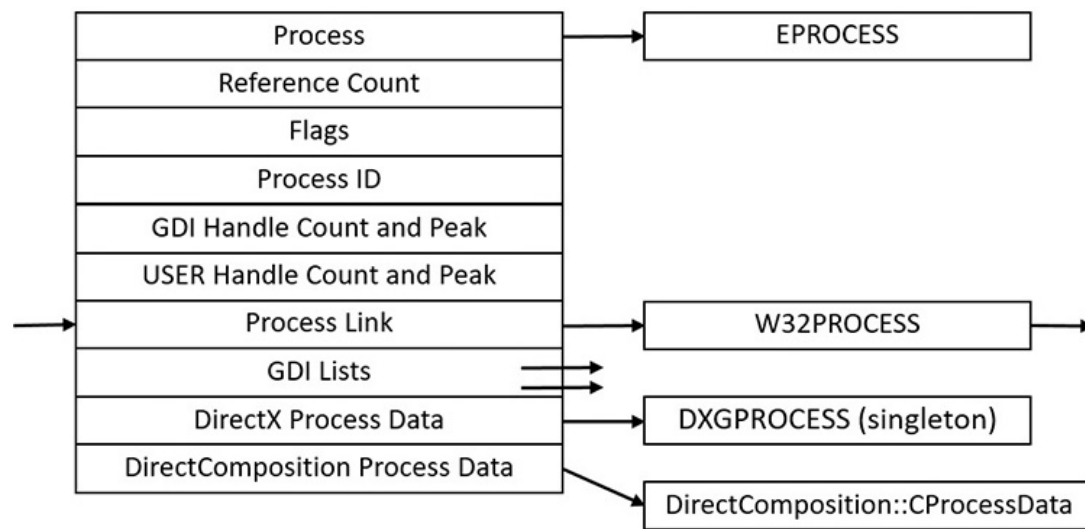


FIGURE 3-6 Fields of the Win32k Process structure.

Protected processes

In the Windows security model, any process running with a token containing the debug privilege (such as an administrator's account) can request any access right that it desires to any other process running on the machine. For example, it can read and write arbitrary process memory, inject code, suspend and resume threads, and query information on other processes. Tools such as Process Explorer and Task Manager need and request these access rights to provide their functionality to users.

This logical behavior (which helps ensure that administrators will always have full control of the running code on the system) clashes with the system behavior for digital rights management requirements imposed by the media industry on computer operating systems that need to support playback of advanced, high-quality digital content such as Blu-ray media. To support reliable and protected playback of such content, Windows Vista and Windows Server 2008 introduced protected processes. These processes exist alongside normal Windows processes, but they add significant constraints to the access rights that other processes on the system (even when running with administrative privileges) can request.

Protected processes can be created by any application. However, the operating system will allow a process to be protected only if the image

file has been digitally signed with a special Windows Media Certificate. The Protected Media Path (PMP) in Windows makes use of protected processes to provide protection for high-value media, and developers of applications such as DVD players can make use of protected processes by using the Media Foundation (MF) API.

The Audio Device Graph process (Audiodg.exe) is a protected process because protected music content can be decoded through it. Related to this is the Media Foundation Protected Pipeline (Mfpmp.exe), which is also a protected process for similar reasons (it does not run by default). Similarly, the Windows Error Reporting (WER; discussed in Chapter 8 in Part 2) client process (Werfaultsecure.exe) can also run protected because it needs to have access to protected processes in case one of them crashes. Finally, the System process itself is protected because some of the decryption information is generated by the Ksecdd.sys driver and stored in its user-mode memory. The System process is also protected to protect the integrity of all kernel handles (because the System process's handle table contains all the kernel handles on the system). Since other drivers may also sometimes map memory inside the user-mode address space of the System process (such as Code Integrity certificate and catalog data), it's yet another reason for keeping the process protected.

At the kernel level, support for protected processes is twofold. First, the bulk of process creation occurs in kernel mode to avoid injection attacks. (The flow for both protected and standard process creation is described in detail in the next section.) Second, protected processes (and their extended cousin, Protected Processes Light [PPL], described in the next section) have special bits set in their `EPROCESS` structure that modify the behavior of security-related routines in the process manager to deny certain access rights that would normally be granted to administrators. In fact, the only access rights that are granted for protected processes are `PROCESS_QUERY/SET_LIMITED_INFORMATION`, `PROCESS_TERMINATE` and `PROCESS_SUSPEND_RESUME`. Certain access rights are also disabled for threads running inside protected processes. We will look at those access rights in [Chapter 4](#) in the section “[Thread internals](#).”

Because Process Explorer uses standard user-mode Windows APIs to query information on process internals, it is unable to perform certain operations on such processes. On the other hand, a tool like WinDbg in kernel-debugging mode, which uses kernel-mode infrastructure to obtain this information, will be able to display complete information. See the experiment in the “[Thread internals](#)” section in [Chapter 4](#) on how Process Explorer behaves when confronted with a protected process such as Audiodg.exe.



As mentioned in [Chapter 1](#), to perform local kernel debugging, you must boot in debugging mode (enabled by using `bcdedit /debug on` or by using the Msconfig advanced boot options). This mitigates against debugger-based attacks on protected processes and the PMP. When booted in debugging mode, high-definition content playback will not work.

Limiting these access rights reliably allows the kernel to sandbox a protected process from user-mode access. On the other hand, because a protected process is indicated by a flag in the `EPROCESS` structure, an administrator can still load a kernel-mode driver that modifies this flag. However, this would be a violation of the PMP model and considered malicious, and such a driver would likely eventually be blocked from loading on a 64-bit system because the kernel-mode, code-signing policy prohibits the digital signing of malicious code. Additionally, kernel-mode patch protection, known as PatchGuard (described in [Chapter 7](#)), as well as the Protected Environment and Authentication Driver (Peauth.sys), will recognize and report such attempts. Even on 32-bit systems, the driver has to be recognized by PMP policy or the playback may be halted. This policy is implemented by Microsoft and not by any kernel detection. This block would require manual action from Microsoft to identify the signature as malicious and update the kernel.

Protected Process Light (PPL)

As we just saw, the original model for protected processes focused on DRM-based content. Starting with Windows 8.1 and Windows Server 2012 R2, an extension to the protected process model was introduced, called Protected Process Light (PPL).

PPLs are protected in the same sense as classic protected processes: User-mode code (even running with elevated privileges) cannot penetrate these processes by injecting threads or obtaining detailed information about loaded DLLs. However, the PPL model adds an additional dimension to the quality of being protected: attribute values. The different Signers have differing trust levels, which in turn results in certain PPLs being more, or less, protected than other PPLs.

Because DRM evolved from merely multimedia DRM to also Windows licensing DRM and Windows Store DRM, standard protected processes are now also differentiated based on the Signer value. Finally, the various recognized Signers also define which access rights are denied to lesser protected processes. For example, normally, the only access masks allowed are `PROCESS_QUERY/SET_LIMITED_INFORMATION` and `PROCESS_SUSPEND_RESUME`. `PROCESS_TERMINATE` is not allowed for certain PPL signers.

Table 3-1 shows the legal values for the protection flag stored in the `EPROCESS` structure.

Internal Protection Process Level Symbol	Protection Type	Signer
PS_PROTECTED_SYSTEM (0x72)	Protected	WinSystem
PS_PROTECTED_WINTCB (0x62)	Protected	WinTcb
PS_PROTECTED_WINTCB_LIGHT (0x61)	Protected Light	WinTcb
PS_PROTECTED_WINDOWS (0x52)	Protected	Windows
PS_PROTECTED_WINDOWS_LIGHT (0x51)	Protected Light	Windows
PS_PROTECTED_LSA_LIGHT (0x41)	Protected Light	Lsa
PS_PROTECTED_ANTIMALWARE_LIGHT (0x31)	Protected Light	Anti-malware
PS_PROTECTED_AUTHENTICODE (0x21)	Protected	Authenticode
PS_PROTECTED_AUTHENTICODE_LIGHT (0x11)	Protected Light	Authenticode
PS_PROTECTED_NONE (0x00)	None	None

TABLE 3-1 Valid protection values for processes

As shown in [Table 3-1](#), there are several signers defined, from high to low power. WinSystem is the highest-priority signer and used for the System process and minimal processes such as the Memory Compression process. For user-mode processes, WinTCB (Windows Trusted Computer Base) is the highest-priority signer and leveraged to protect critical processes that the kernel has intimate knowledge of and might reduce its security boundary toward. When interpreting the power of a process, keep in mind that first, protected processes always trump PPLs, and that next, higher-value signer processes have access to lower ones, but not vice versa. [Table 3-2](#) shows the signer levels (higher values denote the signer is more powerful) and some examples of their usage. You can also dump these in the debugger with the `_PS_PROTECTED_SIGNER` type.

Signer Name (PS_PROTECTED_SIGNER)	Level	Used For
PsProtectedSignerWinSystem	7	System and minimal processes (including Pico processes).
PsProtectedSignerWinTcb	6	Critical Windows components. PROCESS_TERMINATE is denied.
PsProtectedSignerWindows	5	Important Windows components handling sensitive data.
PsProtectedSignerLsa	4	Lsass.exe (if configured to run protected).
PsProtectedSignerAntimalware	3	Anti-malware services and processes, including third party. PROCESS_TERMINATE is denied.
PsProtectedSignerCodeGen	2	NGEN (.NET native code generation).
PsProtectedSignerAuthenticode	1	Hosting DRM content or loading user-mode fonts.
PsProtectedSignerNone	0	Not valid (no protection).

TABLE 3-2 Signers and levels

At this point you may be wondering what prohibits a malicious process from claiming it is a protected process and shielding itself from anti-malware (AM) applications. Because the Windows Media DRM Certificate is no longer necessary to run as a protected process, Microsoft extended its Code Integrity module to understand two special enhanced key usage (EKU) OIDs that can be encoded in a digital code signing certificate: 1.3.6.1.4.1.311.10.3.22 and 1.3.6.1.4.1.311.10.3.20. Once one of these EKUs is present, hardcoded Signer and Issuer strings in the certificate, combined with additional possible EKUs, are then associated with the various Protected Signer values. For example, the Microsoft Windows Issuer can grant the `PsProtectedSignerWindows` protected signer value, but only if the ECU for Windows System Component Verification (1.3.6.1.4.1.311.10.3.6) is also present. As an ex-

ample, [Figure 3-7](#) shows the certificate for Smss.exe, which is permitted to run as WinTcb-Light.

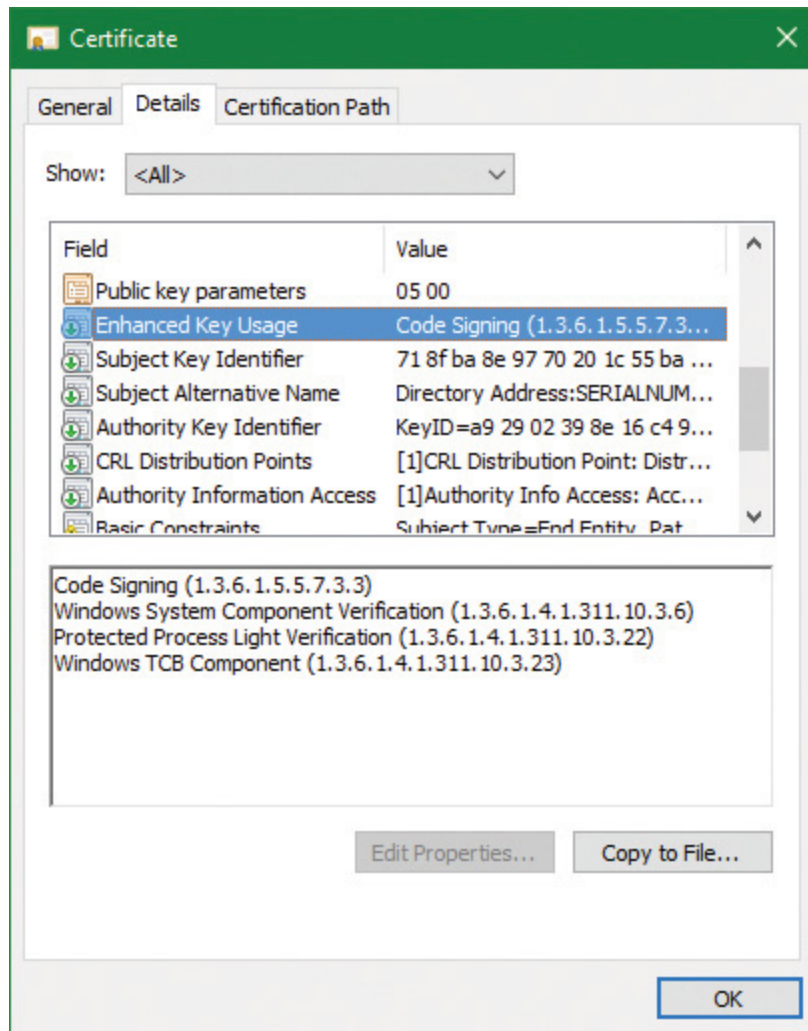


FIGURE 3-7 Smss certificate.

Finally, note that the protection level of a process also impacts which DLLs it will be allowed to load—otherwise, either through a logic bug or simple file replacement or plating, a legitimate protected process could be coerced into loading a third party or malicious library, which would now execute with the same protection level as the process. This check is implemented by granting each process a “Signature Level,” which is stored in the `SignatureLevel` field of `EPROCESS`, and then using an internal lookup table to find a corresponding “DLL Signature Level,” stored as `SectionSignatureLevel` in `EPROCESS`. Any DLL loading in the process will be checked by the Code Integrity component in the same way that the main executable is verified. For example, a process with “WinTcb” as its executable signer will only load “Windows” or higher signed DLLs.

On Windows 10 and Windows Server 2016, the following processes are PPL signed with WinTcb-Lite: smss.exe, csrss.exe, services.exe, and wininit.exe. Lsass.exe is running as PPL on ARM-based Windows (such as Windows mobile 10) and can run as PPL on x86/x64 if configured as such by a registry setting or by policy (see [Chapter 7](#) for more information). Additionally, certain services are configured to run as Windows PPL or protected processes, such as sppsvc.exe (Software Protection Platform). You may also notice certain service-hosting processes (Svchost.exe) running with this protection level, since many services, such as the AppX Deployment Service and the Windows Subsystem for Linux Service, also run protected. More information on such protected services will be described in Chapter 9 in Part 2.

The fact that these core system binaries run as TCB is critical to the security of the system. For example, Csrss.exe has access to certain private APIs implemented by the Window Manager (Win32k.sys), which could give an attacker with Administrator rights access to sensitive parts of the kernel. Similarly, Smss.exe and Wininit.exe implement system startup and management logic that is critical to perform without possible interference from an administrator. Windows guarantees that these binaries will always run as WinTcb-Lite such that, for example, it is not possible for someone to launch them without specifying the correct process protection level in the process attributes when calling `CreateProcess`. This guarantee is known as the *minimum TCB list* and forces any processes with the names in [Table 3-3](#) that are in a System path to have a minimum protection level and/or signing level regardless of the caller's input.

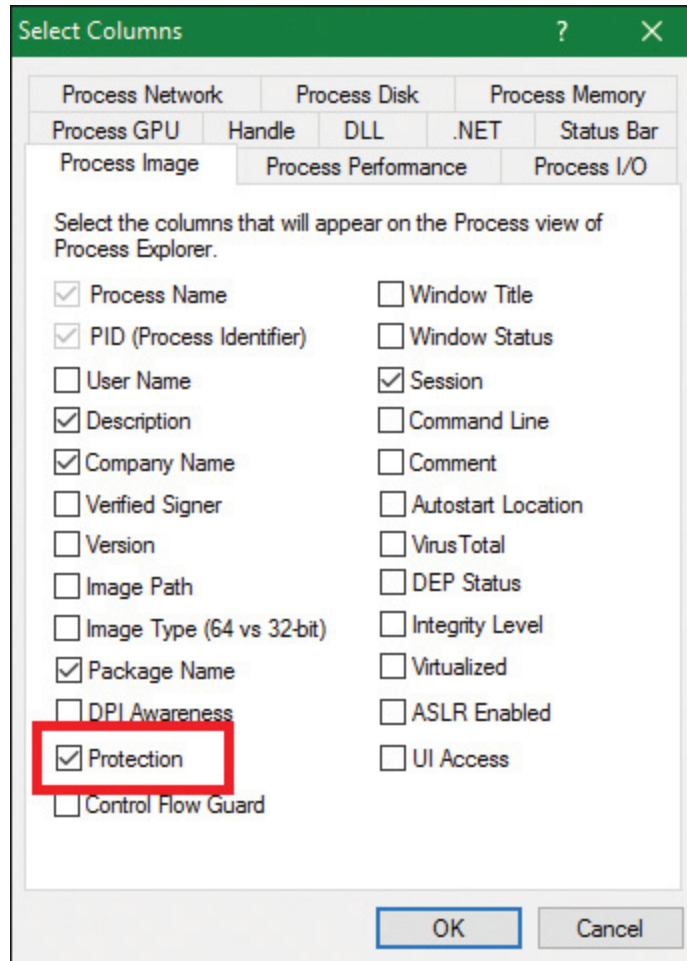
Process Name	Minimum Signature Level	Minimum Protection Level
Smss.exe	Inferred from protection level	WinTcb-Lite
Csrss.exe	Inferred from protection level	WinTcb-Lite
Wininit.exe	Inferred from protection level	WinTcb-Lite
Services.exe	Inferred from protection level	WinTcb-Lite
Werfaultsecure.exe	Inferred from protection level	WinTcb-Full
Sppsvc.exe	Inferred from protection level	Windows-Full
Genvalobj.exe	Inferred from protection level	Windows-Full
Lsass.exe	SE_SIGNING_LEVEL_WINDOWS	0
Userinit.exe	SE_SIGNING_LEVEL_WINDOWS	0
Winlogon.exe	SE_SIGNING_LEVEL_WINDOWS	0
Autochk.exe	SE_SIGNING_LEVEL_WINDOWS*	0

*Only on UEFI firmware systems

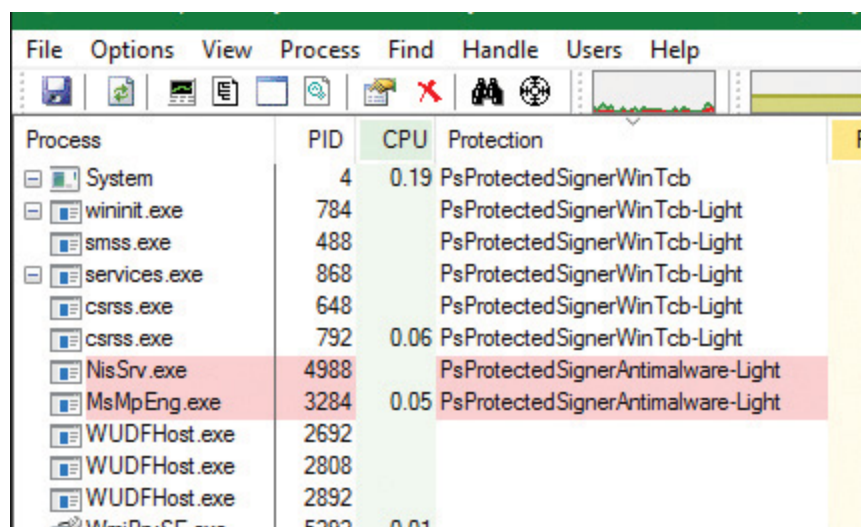
TABLE 3-3 Minimum TCB

EXPERIMENT: VIEWING PROTECTED PROCESSES IN PROCESS EXPLORER

In this experiment, we'll look at how Process Explorer shows protected processes (of either type). Run Process Explorer and select the **Protection** check box in the Process Image tab to view the Protection column:



Now sort by the Protection column in descending order and scroll to the top. You should see all protected processes with their protection type. Here's a screenshot from a Windows 10 x64 machine:



If you select a protected process and look at the lower part when configured to view DLLs, you'll see nothing. That's because Process Explorer uses user-mode APIs to query the loaded modules and that requires access that is not granted for accessing protected processes. The notable exception is the System process, which is protected, but Process Explorer instead shows the list of loaded kernel modules (mostly drivers) since there are no DLLs in system processes. This is done using the `EnumDeviceDrivers` API, which is a system API that does not require a process handle.

If you switch to Handle view, you'll see complete handle information. The reason is similar: Process Explorer uses an undocumented API that returns all handles on the system, which does not require a specific process handle. Process Explorer can identify the process simply because this information returns the PID associated with each handle.

Third-party PPL support

The PPL mechanism extends the protection possibilities for processes beyond executables created solely by Microsoft. A common example is anti-malware (AM) software. A typical AM product consists of three main components:

- A kernel driver that intercepts I/O requests to the file system and/or the network, and implements blocking capabilities using object, process, and thread callbacks
- A user-mode service (typically running under a privileged account) that configures the driver's policies, receives notifications from the driver regarding "interesting" events (for example, infected file), and may communicate with a local server or the Internet
- A user-mode GUI process that communicates information to the user and optionally allows the user to make decisions where applicable.

One possible way malware can attack a system is by managing to inject code inside a process running with elevated privileges, or better, inject code specifically inside an anti-malware service and thus tamper with it or disable its operation. If, however, the AM service could run as a PPL, no code injection would be possible, and no process termination would be allowed, meaning that the AM software would be better protected from malware that does not employ kernel-level exploits.

To enable this use, the AM kernel driver described above needs to have a corresponding Early-Launch Anti Malware (ELAM) driver. While ELAM is further described in [Chapter 7](#), the key distinction is that such drivers require a special anti-malware certificate provided by Microsoft (after proper verification of the software's publisher). Once such a driver is installed, it can contain a custom resource section in its main executable (PE) file called `ELAMCERTIFICATEINFO`. This section can describe three additional Signers (identified by their public key), each having up to three additional EKUs (identified by OID). Once the Code Integrity system recognizes any file signed by one of the three Signers, containing one of the three EKUs, it permits the process to request a PPL of `PS_PROTECTED_ANTIMALWARE_LIGHT (0x31)`. A canonical example of this is Microsoft's own AM known as Windows Defender. Its service on Windows 10 (MsMpEng.exe) is signed with the anti-malware certificate for better protection against malware attacking the AM itself, as is its Network Inspection Server (NisSvc.exe).

Minimal and Pico processes

The types of processes we've looked at so far, and their data structures, seem to imply that their use is the execution of user-mode code, and that they contain a great deal of related data structures in memory to achieve this. Yet, not all processes are used for this purpose. For example, as we've seen, the System process is merely used as a container of most of the system threads, such that their execution time doesn't pollute arbitrary user-mode processes, as well as being used as a container of drivers' handles (called kernel handles), such that these don't end up owned by an arbitrary application either.

Minimal processes

When a specific flag is given to the `NtCreateProcessEx` function, and the caller is kernel-mode, the function behaves slightly differently and causes the execution of the `PsCreateMinimalProcess` API. In turn, this causes a process to be created without many of the structures that we saw earlier, namely:

- No user-mode address space will be set up, so no PEB and related structures will exist.
- No NTDLL will be mapped into the process, nor will any loader/API Set information.
- No section object will be tied to the process, meaning no executable image file is associated to its execution or its name (which can be empty, or an arbitrary string).
- The `Minimal` flag will be set in the `EPROCESS` flags, causing all threads to become minimal threads, and also avoid any user-mode allocations such as their TEB or user-mode stack. (See [Chapter 4](#) for more information on the TEB.)

As we saw in [Chapter 2](#), Windows 10 has at least two minimal processes—the System process and Memory Compression process—and can have a third, the Secure System process, if Virtualization-Based Security is enabled, which is described further in [Chapter 2](#) and [Chapter 7](#).

Finally, the other way to have minimal processes running on a Windows 10 system is to enable the Windows Subsystem for Linux (WSL) optional feature that was also described in [Chapter 2](#). This will install an inbox Pico Provider composed of the `Lxss.sys` and `LxCore.sys` drivers.

Pico processes

While minimal processes have a limited use in terms of allowing access to user-mode virtual address space from kernel components and protecting it, Pico processes take on a more important role by permitting a special component, called a *Pico Provider*, to control most aspects of their execution from an operating system perspective. This level of control ultimately allows such a provider to emulate the behavior of a completely different operating system kernel, without the underlying user-mode binary being aware that it is running on a Windows-based operating system. This is essentially an implementation of the Drawbridge project from Microsoft Research, which is also used to support SQL Server for Linux in a similar way (albeit with a Windows-based Library OS on top of the Linux kernel).

To support the existence of Pico processes on the system, a provider must first be present. Such a provider can be registered with the `PsRegisterPicoProvider` API, but subject to a very specific rule: A Pico provider must be loaded before any other third-party drivers are loaded (including boot drivers). In fact, only one of the limited set of a dozen or so core drivers are allowed to call this API before the functionality is disabled, and these core drivers must be signed with a Microsoft Signer Certificate and Windows Component ECU. On Windows systems with the optional WSL component enabled, this core driver is called `Lxss.sys`, and serves as a stub driver until another driver, `LxCore.sys`, loads a bit later and takes over the Pico provider responsibilities by transferring the various dispatch tables over to itself. Additionally, note that at the time of this writing, only one such core driver can register itself as a Pico provider.

When a Pico provider calls the registration API, it receives a set of function pointers, which allow it to create and manage Pico processes:

- One function to create a Pico process and one to create a Pico thread.
- One function to get the context (an arbitrary pointer that the provider can use to store specific data) of a Pico process, one to set it, and an-

other pair of functions to do the same for Pico threads. This will populate the `PicoContext` field in `ETHREAD` and/or `EPROCESS`.

- One function to get the CPU context structure (`CONTEXT`) of a Pico thread and one to set it.
- A function to change the `FS` and/or `GS` segments of a Pico thread, which are normally used by user-mode code to point to some thread local structure (such as the TEB on Windows).
- One function to terminate a Pico thread and one to do the same to a Pico process.
- One function to suspend a Pico thread and one to resume it.

As you can see, through these functions, the Pico provider can now create fully custom processes and threads for whom it controls the initial starting state, segment registers, and associate data. However, this alone would not allow the ability to emulate another operating system. A second set of function pointers is transferred, this time from the provider to the kernel, which serve as callbacks whenever certain activities of interest will be performed by a Pico thread or process.

- A callback whenever a Pico thread makes a system call using the `SYSCALL` instruction
- A callback whenever an exception is raised from a Pico thread
- A callback whenever a fault during a probe and lock operation on a memory descriptor list (MDL) occurs inside a Pico thread
- A callback whenever a caller is requesting the name of a Pico process
- A callback whenever Event Tracing for Windows (ETW) is requesting the user-mode stack trace of a Pico process

- A callback whenever an application attempts to open a handle to a Pico process or Pico thread
- A callback whenever someone requests the termination of a Pico process
- A callback whenever a Pico thread or Pico process terminates unexpectedly

Additionally, a Pico provider also leverages Kernel Patch Protection (KPP), described in [Chapter 7](#), to both protect its callbacks and system calls as well as prevent fraudulent or malicious Pico providers from registering on top of a legitimate Pico provider.

It now becomes clear that with such unparalleled access to any possible user-kernel transition or visible kernel-user interactions between a Pico process/thread and the world, it can be fully encapsulated by a Pico provider (and relevant user-mode libraries) to wrap a completely different kernel implementation than that of Windows (with some exceptions, of course, as thread scheduling rules and memory management rules, such as commit, still apply). Correctly written applications are not supposed to be sensitive to such internal algorithms, as they are subject to change even within the operating system they normally execute on.

Therefore, Pico providers are essentially custom-written kernel modules that implement the necessary callbacks to respond to the list of possible events (shown earlier) that a Pico process can cause to arise. This is how WSL is capable of running unmodified Linux ELF binaries in user-mode, limited only by the completeness of its system call emulation and related functionality.

To complete the picture on regular NT processes versus minimal processes versus Pico processes, we present [Figure 3-8](#), showing the different structures for each.

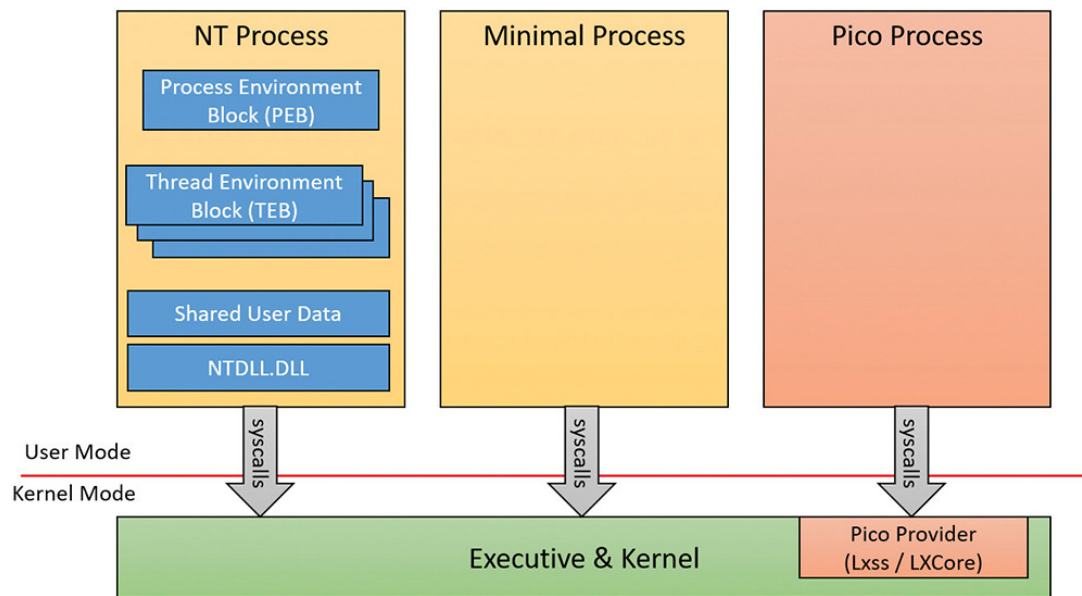


FIGURE 3-8 Process types.

Trustlets (secure processes)

As covered in [Chapter 2](#), Windows contains new virtualization-based security (VBS) features such as Device Guard and Credential Guard, which enhance the safety of the operating system and user data by leveraging the hypervisor. We saw how one such feature, Credential Guard (which is discussed at length in [Chapter 7](#)), runs in a new Isolated User Mode environment, which, while still unprivileged (ring 3), has a virtual trust level of 1 (VTL 1), granting it protection from the regular VTL 0 world in which both the NT kernel (ring 0) and applications (ring 3) live. Let's investigate how the kernel sets up such processes for execution, and the various data structures such processes use.

Trustlet structure

To begin with, although Trustlets are regular Windows Portable Executables (PE) files, they contain some IUM-specific properties:

- They can import only from a limited set of Windows system DLLs (C/C++ Runtime, KernelBase, Advapi, RPC Runtime, CNG Base Crypto, and NTDLL) due to the restricted number of system calls that are available to Trustlets. Note that mathematical DLLs that operate only on

data structures (such as NTLM, ASN.1, etc.) are also usable, as they don't perform any system calls.

- They can import from an IUM-specific system DLL that is made available to them, called `Iumbase`, which provides the Base IUM System API, containing support for mailslots, storage boxes, cryptography, and more. This library ends up calling into `Iumdll.dll`, which is the VTL 1 version of `Ntdll.dll`, and contains secure system calls (system calls that are implemented by the Secure Kernel, and not passed on to the Normal VTL 0 Kernel).

- They contain a PE section named `.tPolicy` with an exported global variable named `s_IumPolicyMetadata`. This serves as metadata for the Secure Kernel to implement policy settings around permitting VTL 0 access to the Trustlet (such as allowing debugging, crash dump support, etc.).

- They are signed with a certificate that contains the Isolated User Mode EKU (1.3.6.1.4.1.311.10.3.37). [Figure 3-9](#) shows the certificate data for `LsaIso.exe`, showing its IUM EKU.

Additionally, Trustlets must be launched by using a specific process attribute when using `CreateProcess`—both to request their execution in IUM as well as to specify launch properties. We will describe both the policy metadata and the process attributes in the following sections.

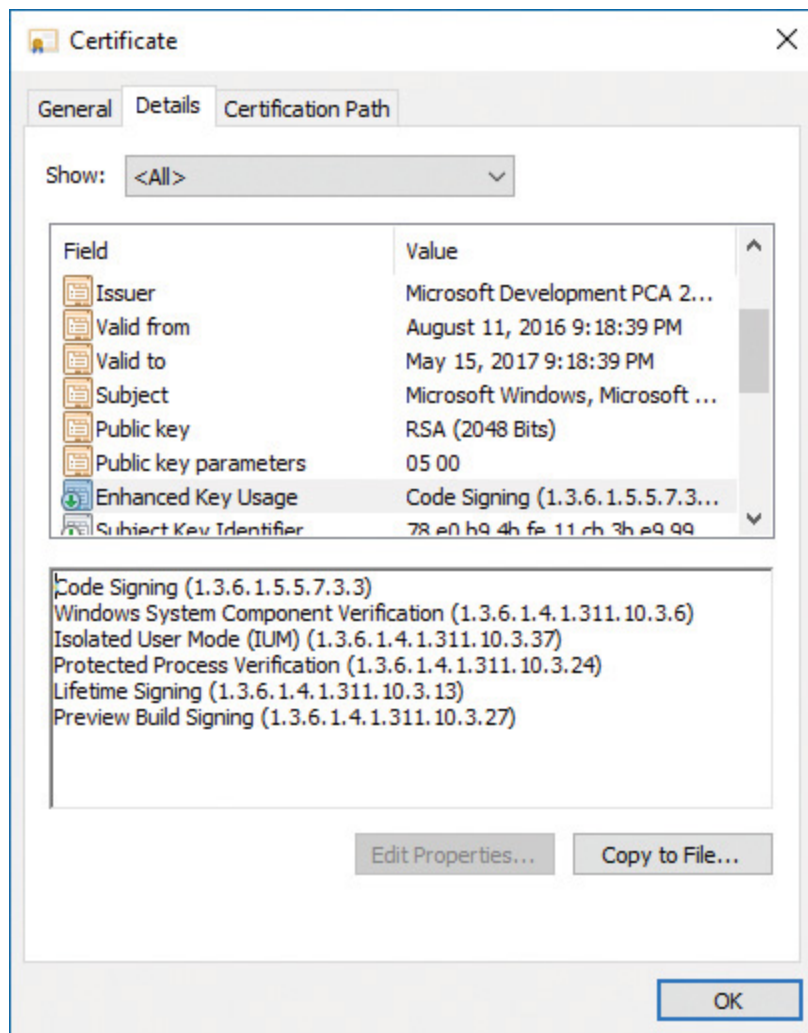


FIGURE 3-9 Trustlet EKU in the certificate.

Trustlet policy metadata

The policy metadata includes various options for configuring how “accessible” the Trustlet will be from VTL 0. It is described by a structure present at the `s_IumPolicyMetadata` export mentioned earlier, and contains a version number (currently set to 1) as well as the Trustlet ID, which is a unique number that identifies this specific Trustlet among the ones that are known to exist (for example, BioIso.exe is Trustlet ID 4). Finally, the metadata has an array of policy options. Currently, the options listed in [Table 3-4](#) are supported. It should be obvious that as these policies are part of the signed executable data, attempting to modify them would invalidate the IUM signature and prohibit execution.

Policy	Meaning	More Information
ETW	Enables or Disables ETW	
Debug	Configures debugging	Debug can be enabled at all times, only when SecureBoot is disabled, or using an on-demand challenge/response mechanism.
Crash Dump	Enables or disables crash dump	
Crash Dump Key	Specifies Public Key for Encrypting Crash Dump	Dumps can be submitted to Microsoft Product Team, which has the private key for decryption
Crash Dump GUID	Specifies identifier for crash dump key	This allows multiple keys to be used/identified by the product team.
Parent Security Descriptor	SDDL format	This is used to validate that the owner/parent process is expected.
Parent Security Descriptor Revision	SDDL format revision ID	This is used to validate that the owner/parent process is expected.
SVN	Security version	This is a unique number that can be used by the Trustlet (along its identity) when encrypting AES256/GCM messages.
Device ID	Secure device PCI identifier	The Trustlet can only communicate with a Secure Device whose PCI ID matches.
Capability	Enables powerful VTL 1 capabilities	This enables access to the Create Secure Section API, DMA and user-mode MMIO access to Secure Devices, and Secure Storage APIs.
Scenario ID	Specifies the scenario ID for this binary	Encoded as a GUID, this must be specified by Trustlets when creating secure image sections to ensure it is for a known scenario.

TABLE 3-4 Trustlet policy options

Trustlet attributes

Launching a Trustlet requires correct usage of the `PS_CP_SECURE_PROCESS` attribute, which is first used to authenticate that the caller truly wants to create a Trustlet, as well as to verify that the Trustlet the caller *thinks* its executing is actually the Trustlet being executed. This is done by embedding a Trustlet identifier in the attribute, which must match the Trustlet ID contained in the policy metadata. Then, one or more attributes can be specified, which are shown in **Table 3-5**.

Attribute	Meaning	More Information
Mailbox Key	Used to retrieve mailbox data	Mailboxes allow the Trustlet to share data with the VTL 0 world as long as the Trustlet key is known.
Collaboration ID	Sets the collaboration ID to use when using the Secure Storage IUM API	Secure Storage allows Trustlets to share data among each other, as long as they have the same collaboration ID. If no collaboration ID is present, the Trustlet instance ID will be used instead.
TK Session ID	Identifies the session ID used during Crypto	

TABLE 3-5 Trustlet attributes

System built-in Trustlets

At the time of this writing, Windows 10 contains five different Trustlets, which are identified by their identity numbers. They are described in **Table 3-6**. Note that Trustlet ID 0 represents the Secure Kernel itself.

Binary Name (Trustlet ID)	Description	Policy Options
Lsalso.exe (1)	Credential and Key Guard Trustlet	Allow ETW, Disable Debugging, Allow Encrypted Crash Dump
Vmosp.exe (2)	Secure Virtual Machine Worker (vTPM Trustlet)	Allow ETW, Disable Debugging, Disable Crash Dump, Enable Secure Storage Capability, Verify Parent Security Descriptor is S-1-5-83-0 (NT VIRTUAL MACHINE\Virtual Machines)
Unknown (3)	vTPM Key Enrollment Trustlet	Unknown
Biolso.exe (4)	Secure Biometrics Trustlet	Allow ETW, Disable Debugging, Allow Encrypted Crash Dump
FsIso.exe (5)	Secure Frame Server Trustlet	Disable ETW, Allow Debugging, Enable Create Secure Section Capability, Use Scenario ID {AE53FC6E-8D89-4488-9D2E-4D008731C5FD}

TABLE 3-6 Built-in Trustlets

Trustlet identity

Trustlets have multiple forms of identity that they can use on the system:

■ **Trustlet identifier or Trustlet ID** This is a hard-coded integer in the Trustlet's policy metadata, which also must be used in the Trustlet process-creation attributes. It ensures that the system knows there are only a handful of Trustlets, and that the callers are launching the expected one.

■ **Trustlet instance** This is a cryptographically secure 16-byte random number generated by the Secure Kernel. Without the use of a collaboration ID, the Trustlet instance is what's used to guarantee that Secure Storage APIs will only allow this one instance of the Trustlet to get/put data into its storage blob.

■ **Collaboration ID** This is used when a Trustlet would like to allow other Trustlets with the same ID, or other instances of the same Trustlet, to share access to the same Secure Storage blob. When this ID is present, the instance ID of the Trustlet will be ignored when calling the Get or Put APIs.

■ **Security version (SVN)** This is used for Trustlets that require strong cryptographic proof of provenance of signed or encrypted data. It is used when encrypting AES256/GCM data by Credential and Key Guard, and is also used by the Cryptograph Report service.

■ **Scenario ID** This is used for Trustlets that create named (identity-based) secure kernel objects, such as secure sections. This GUID validates that the Trustlet is creating such objects as part of a predetermined scenario, by tagging them in the namespace with this GUID. As such, other Trustlets wishing to open the same named objects would thus have to have the same scenario ID. Note that more than one scenario ID can actually be present, but no Trustlets currently use more than one.

Isolated user-mode services

The benefits of running as a Trustlet not only include protection from attacks from the normal (VTL 0) world, but also access to privileged and protected secure system calls that are only offered by the Secure Kernel to Trustlets. These include the following services:

■ **Secure Devices (IumCreateSecureDevice, IumDmaMapMemory, IumGetDmaEnabler, IumMap-SecureIo, IumProtectSecureIo, IumQuerySecureDeviceInformation, IopUnmapSecureIo, IumUpdateSecureDeviceState)** These provide access to secure ACPI and/or PCI devices, which cannot be accessed from VTL 0 and are exclusively owned by the Secure Kernel (and its ancillary Secure HAL and Secure PCI services). Trustlets with the relevant capabilities (see the “[Trustlet policy metadata](#)” section earlier in this chapter) can map the registers of such a device in VTL 1 IUM, as well as potentially perform Direct Memory Access (DMA) transfers. Additionally, Trustlets can serve as user-mode device drivers for such hardware by using the Secure Device Framework (SDF) located in SDFHost.dll. This functionality is leveraged for Secure Biometrics for Windows Hello, such as Secure USB Smartcard (over PCI) or Webcam/Fingerprint Sensors (over ACPI).

■ **Secure Sections (`IumCreateSecureSection`, `IumFlushSecureSectionBuffers`, `IumGetExposed-SecureSection`, `IumOpenSecureSection`)** These provide the ability to both share physical pages with a VTL 0 driver (which would use `VslCreateSecureSection`) through exposed secure sections, as well as share data solely within VTL 1 as named secured sections (leveraging the identity-based mechanism described earlier in the “[Trustlet identity](#)” section) with other Trustlets or other instances of the same Trustlet. Trustlets require the Secure Section capability described in the “[Trustlet policy metadata](#)” section to use these features.

■ **Mailboxes (`IumPostMailbox`)** This enables a Trustlet to share up to eight slots of about up to 4 KB of data with a component in the normal (VTL 0) kernel, which can call `VslRetrieveMailbox` passing in the slot identifier and secret mailbox key. For example, Vid.sys in VTL 0 uses this to retrieve various secrets used by the vTPM feature from the Vmsp.exe Trustlet.

■ **Identity Keys (`IumGetIdk`)** This allows a Trustlet to obtain either a unique identifying decryption key or signing key. This key material is unique to the machine and can be obtained only from a Trustlet. It is an essential part of the Credential Guard feature to uniquely authenticate the machine and that credentials are coming from IUM.

■ **Cryptographic Services (`IumCrypto`)** This allows a Trustlet to encrypt and decrypt data with a local and/or per-boot session key generated by the Secure Kernel that is only available to IUM, to obtain a TPM binding handle, to get the FIPS mode of the Secure Kernel, and to obtain a random number generator (RNG) seed only generated by the Secure Kernel for IUM. It also enables a Trustlet to generate an IDK-signed, SHA-2 hashed, and timestamped report with the identity and SVN of the Trustlet, a dump of its policy metadata, whether or not it was ever attached to a debugger, and any other Trustlet-controlled data requested. This can be used as a sort of TPM-like measurement of the Trustlet to prove that it was not tampered with.

■ **Secure Storage (`IumSecureStorageGet`, `IumSecureStoragePut`)** This allows Trustlets that have the Secure Storage capability (described earlier in the “[Trustlet policy metadata](#)” section) to store arbitrarily sized storage blobs and to later retrieve them, either based on their unique Trustlet instance or by sharing the same collaboration ID as another Trustlet.

Trustlet-accessible system calls

As the Secure Kernel attempts to minimize its attack surface and exposure, it only provides a subset (less than 50) of all of the hundreds of system calls that a normal (VTL 0) application can use. These system calls are the strict minimum necessary for compatibility with the system DLLs that Trustlets can use (refer to the section “[Trustlet structure](#)” to see these), as well as the specific services required to support the RPC runtime (`Rpcrt4.dll`) and ETW tracing.

■ **Worker Factory and Thread APIs** These support the Thread Pool API (used by RPC) and TLS Slots used by the Loader.

■ **Process Information API** This supports TLS Slots and Thread Stack Allocation.

■ **Event, Semaphore, Wait, and Completion APIs** These support Thread Pool and Synchronization.

■ **Advanced Local Procedure Call (ALPC) APIs** These support Local RPC over the `ncalrpc` transport.

■ **System Information API** This supports reading Secure Boot information, basic and NUMA system information for `Kernel32.dll` and Thread Pool scaling, performance, and subsets of time information.

■ **Token API** This provides minimal support for RPC impersonation.

■ **Virtual Memory Allocation APIs** These support allocations by the User-Mode Heap Manager.

■ **Section APIs** These support the Loader (for DLL Images) as well as the Secure Section functionality (once created/exposed through secure system calls shown earlier).

■ **Trace Control API** This supports ETW.

■ **Exception and Continue API** This supports Structured Exception Handling (SEH).

It should be evident from this list that support for operations such as Device I/O, whether on files or actual physical devices, is not possible (there is no `CreateFile` API, to begin with), as is also the case for Registry I/O. Nor is the creation of other processes, or any sort of graphics API usage (there is no Win32k.sys driver in VTL 1). As such, Trustlets are meant to be isolated workhorse back-ends (in VTL 1) of their complex front-ends (in VTL 0), having only ALPC as a communication mechanism, or exposed secure sections (whose handle would have to have been communicated to them through ALPC). In [Chapter 7](#) (Security), we'll look in more detail into the implementation of a specific Trustlet—LsaIso.exe, which provides Credential and Key Guard.

EXPERIMENT: IDENTIFYING SECURE PROCESSES

Secure processes, other than being known by their name, can be identified in the kernel debugger in two ways. First, each secure process has a secure PID, which represents its handle in the Secure Kernel's handle table. This is used by the normal (VTL 0) kernel when creating threads in the process or requesting its termination. Secondly, the threads themselves have a thread cookie associated with them, which represents their index in the Secure Kernel's thread table.

You can try the following in a kernel debugger:

[Click here to view code image](#)

```
lkd> !for_each_process .if @@@(((nt!_EPROCESS*)${@#Process})-
>Pcb.SecurePid) {
.printf "Trustlet: %ma (%p)\n", @@@(((nt!_EPROCESS*)${@#Process})-
>ImageFileName),
@#Process }
Trustlet: Secure System (ffff9b09d8c79080)
Trustlet: LsaIso.exe (ffff9b09e2ba9640)
Trustlet: BioIso.exe (ffff9b09e61c4640)
lkd> dt nt!_EPROCESS ffff9b09d8c79080 Pcb.SecurePid
+0x000 Pcb      :
+0x2d0 SecurePid : 0x00000001'40000004
lkd> dt nt!_EPROCESS ffff9b09e2ba9640 Pcb.SecurePid
+0x000 Pcb      :
+0x2d0 SecurePid : 0x00000001'40000030
lkd> dt nt!_EPROCESS ffff9b09e61c4640 Pcb.SecurePid
+0x000 Pcb      :
+0x2d0 SecurePid : 0x00000001'40000080
lkd> !process ffff9b09e2ba9640 4
PROCESS ffff9b09e2ba9640
  SessionId: 0 Cid: 0388 Peb: 6cdc62b000 ParentCid: 0328
  DirBase: 2f254000 ObjectTable: ffffc607b59b1040 HandleCount: 44.
  Image: LsaIso.exe
  THREAD ffff9b09e2ba2080 Cid 0388.038c Teb: 00000006cdc62c000
```

Win32Thread:

0000000000000000 WAIT

lkd> dt nt!_ETHREAD ffff9b09e2ba2080 Tcb.SecureThreadCookie

+0x000 Tcb :

+0x31c SecureThreadCookie : 9

Flow of CreateProcess

We've shown the various data structures involved in process-state manipulation and management and how various tools and debugger commands can inspect this information. In this section, we'll see how and when those data structures are created and filled out, as well as the overall creation and termination behaviors behind processes. As we've seen, all documented process-creation functions eventually end up calling `CreateProcessInternalW`, so this is where we start.

Creating a Windows process consists of several stages carried out in three parts of the operating system: the Windows client-side library `Kernel32.dll` (the real work starting with `CreateProcessInternalW`), the Windows executive, and the Windows subsystem process (`Csrss`). Because of the multiple-environment subsystem architecture of Windows, creating an executive process object (which other subsystems can use) is separated from the work involved in creating a Windows subsystem process. So, although the following description of the flow of the Windows `CreateProcess` function is complicated, keep in mind that part of the work is specific to the semantics added by the Windows subsystem as opposed to the core work needed to create an executive process object.

The following list summarizes the main stages of creating a process with the Windows `CreateProcess*` functions. The operations performed in each stage are described in detail in the subsequent sections.



NOTE

Many steps of `CreateProcess` are related to the setup of the process virtual address space and therefore refer to many memory-management terms and structures that are defined in [Chapter 5](#).

1. Validate parameters; convert Windows subsystem flags and options to their native counterparts; parse, validate, and convert the attribute list to its native counterpart.
2. Open the image file (.exe) to be executed inside the process.
3. Create the Windows executive process object.
4. Create the initial thread (stack, context, and Windows executive thread object).
5. Perform post-creation, Windows subsystem-specific process initialization.
6. Start execution of the initial thread (unless the `CREATE_SUSPENDED` flag was specified).
7. In the context of the new process and thread, complete the initialization of the address space (for example, load required DLLs) and begin execution of the program's entry point.

[Figure 3-10](#) shows an overview of the stages Windows follows to create a process.

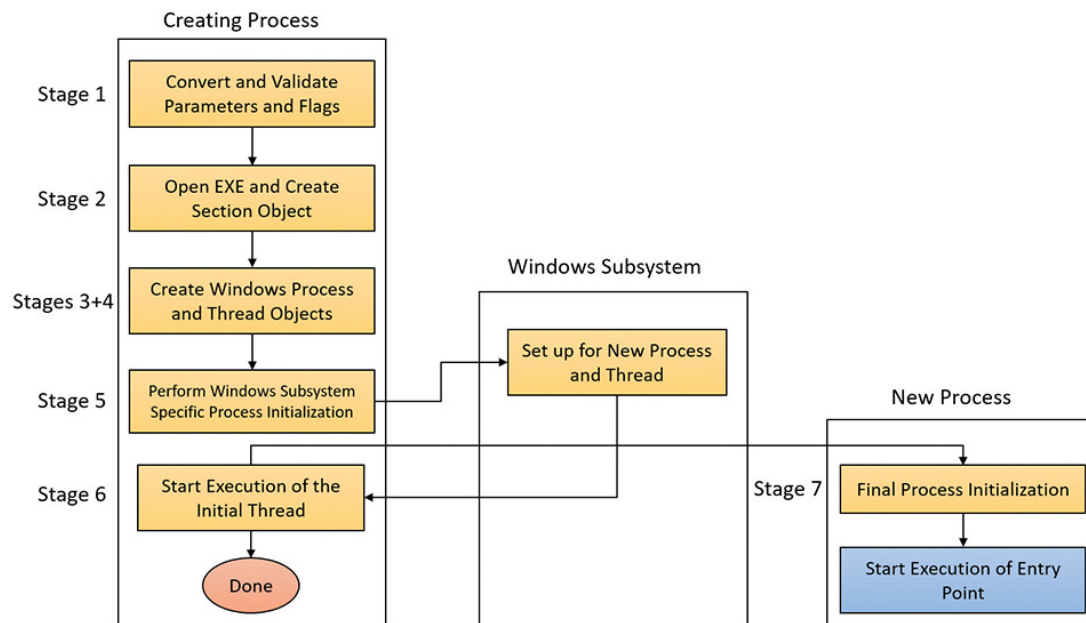


FIGURE 3-10 The main stages of process creation.

Stage 1: Converting and validating parameters and flags

Before opening the executable image to run, `CreateProcessInternalW` performs the following steps:

1. The priority class for the new process is specified as independent bits in the `CreationFlags` parameter to the `CreateProcess*` functions. Thus, you can specify more than one priority class for a single `CreateProcess*` call. Windows resolves the question of which priority class to assign to the process by choosing the lowest-priority class set.

There are six process priority classes defined, each value mapped to a number:

- Idle or Low, as Task Manager displays it (4)
- Below Normal (6)
- Normal (8)
- Above Normal (10)
- High (13)
- Real-time (24)

The priority class is used as the base priority for threads created in that process. This value does not directly affect the process itself—only the threads inside it. A description of process priority class and its effects on thread scheduling appears in [Chapter 4](#).

2. If no priority class is specified for the new process, the priority class defaults to Normal. If a Real-time priority class is specified for the new process and the process's caller doesn't have the Increase Scheduling Priority privilege (`SE_INC_BASE_PRIORITY_NAME`), the High priority class is used instead. In other words, process creation doesn't fail just because the caller has insufficient privileges to create the process in the Real-time priority class; the new process just won't have as high a priority as Real-time.

3. If the creation flags specify that the process will be debugged, Kernel32 initiates a connection to the native debugging code in Ntdll.dll by calling `DbgUiConnectToDbg` and gets a handle to the debug object from the current thread's environment block (TEB).

4. Kernel32.dll sets the default hard error mode if the creation flags specified one.

5. The user-specified attribute list is converted from Windows subsystem format to native format and internal attributes are added to it. The possible attributes that can be added to the attribute list are listed in [Table 3-7](#), including their documented Windows API counterparts, if any.

Native Attribute	Equivalent Win32 Attribute	Type	Description
PS_CP_PARENT_PROCESS	PROC_THREAD_ATTRIBUTE_PARENT_PROCESS. Also used when elevating.	Input	Handle to the parent process.
PS_CP_DEBUG_OBJECT	N/A. Used when using DEBUG_PROCESS as a flag.	Input	Debug object if process is being started debugged.
PS_CP_PRIMARY_TOKEN	N/A. Used when using CreateProcessAsUser/WithTokenW.	Input	Process token if CreateProcessAsUser was used.
PS_CP_CLIENT_ID	N/A. Returned by Win32 API as a parameter (PROCESS_INFORMATION).	Output	Returns the TID and PID of the initial thread and the process.
PS_CP_TEB_ADDRESS	N/A. Internally used and not exposed.	Output	Returns the address of the TEB for the initial thread.
PS_CP_FILENAME	N/A. Used as a parameter in CreateProcess APIs.	Input	The name of the process that should be created.
PS_CP_IMAGE_INFO	N/A. Internally used and not exposed.	Output	Returns SECTION_IMAGE_INFORMATION, which contains information on the version, flags, and subsystem of the executable, as well as the stack size and entry point.
PS_CP_MEM_RESERVE	N/A. Internally used by SMSS and CSRSS.	Input	An array of virtual memory reservations that should be made during initial process address space creation, allowing guaranteed availability because no other allocations have taken place yet.
PS_CP_PRIORITY_CLASS	N/A. Passed in as a parameter to the CreateProcess API.	Input	Priority class that the process should be given.
PS_CP_ERROR_MODE	N/A. Passed in through the CREATE_DEFAULT_ERROR_MODE flag.	Input	Hard error-processing mode for the process.
PS_CP_STD_HANDLE_INFO	None. Used internally.	Input	Specifies whether standard handles should be duplicated or new handles should be created.
PS_CP_HANDLE_LIST	PROC_THREAD_ATTRIBUTE_HANDLE_LIST	Input	A list of handles belonging to the parent process that should be inherited by the new process.
PS_CP_GROUP_AFFINITY	PROC_THREAD_ATTRIBUTE_GROUP_AFFINITY	Input	Processor group(s) the thread should be allowed to run on.
PS_CP_PREFERRED_NODE	PROC_THREAD_ATTRIBUTES_PREFERRED_NODE	Input	The preferred (ideal) NUMA node that should be associated with the process. It affects the node on which the initial process heap and thread stack will be created (see Chapter 5).
PS_CP_IDEAL_PROCESSOR	PROC_THREAD_ATTRIBUTE_IDEAL_PROCESSOR	Input	The preferred (ideal) processor that the thread should be scheduled on.
PS_CP_UMS_THREAD	PROC_THREAD_ATTRIBUTE_UMS_THREAD	Input	Contains the UMS attributes, completion list, and context.
PS_CP_MITIGATION_OPTIONS	PROC_THREAD_MITIGATION_POLICY	Input	Contains information on which mitigations (SEHOP, ATL Emulation, NX) should be enabled/disabled for the process.
PS_CP_PROTECTION_LEVEL	PROC_THREAD_ATTRIBUTE_PROTECTION_LEVEL	Input	Must point to one of the allowed process protection values shown in Table 3-1 or the value PROTECT_LEVEL_SAME to indicate the same protection level as the parent.

PS_CP_SECURE_PROCESS	None. Used internally.	Input	Indicates the process should run as an Isolated User Mode (IUM) Trustlet. See Chapter 8 in Part 2 for more details.
PS_CP_JOB_LIST	None. Used internally.	Input	Assigns the process to a list of jobs.
PS_CP_CHILD_PROCESS_POLICY	PROC_THREAD_ATTRIBUTE_CHILD_PROCESS_POLICY	Input	Specifies whether the new process is allowed to create child processes, either directly or indirectly (such as by using WMI).
PS_CP_ALL_APPLICATION_PACKAGES_POLICY	PROC_THREAD_ATTRIBUTE_ALL_APPLICATION_PACKAGES_POLICY	Input	Specifies if the AppContainer token should be excluded from ACL checks that include the ALL APPLICATION PACKAGES group. The ALL RESTRICTED APPLICATION PACKAGES group will be used instead.
PS_CP_WIN32K_FILTER	PROC_THREAD_ATTRIBUTE_WIN32K_FILTER	Input	Indicates if the process will have many of its GDI/USER system calls to Win32k.sys filtered out (blocked), or if they will be permitted but audited. Used by the Microsoft Edge browser to reduce attack surface.
PS_CP_SAFE_OPEN_PROMPT_ORIGIN_CLAIM	None. Used internally.	Input	Used by the Mark of the Web functionality to indicate the file came from an untrusted source.
PS_CP_BNO_ISOLATION	PROC_THREAD_ATTRIBUTE_BNO_ISOLATION	Input	Causes the primary token of the process to be associated with an isolated BaseNamedObjects directory. (See Chapter 8 in Part 2 for more information on named objects.)
PS_CP_DESKTOP_APP_POLICY	PROC_THREAD_ATTRIBUTE_DESKTOP_APP_POLICY	Input	Indicates if the modern application will be allowed to launch legacy desktop applications, and if so, in what way.
None—used internally	PROC_THREAD_ATTRIBUTE_SECURITY_CAPABILITIES	Input	Specifies a pointer to a SECURITY_CAPABILITIES structure, which is used to create the AppContainer token for the process before calling NtCreateUserProcess.

TABLE 3-7 Process attributes



NOTE

The attribute list passed on `CreateProcess*` calls permits passing back to the caller information beyond a simple status code, such as the TEB address of the initial thread or information on the image section. This is necessary for protected processes because the parent cannot query this information after the child is created.

6. If the process is part of a job object, but the creation flags requested a separate virtual DOS machine (VDM), the flag is ignored.

7. The security attributes for the process and initial thread that were supplied to the `CreateProcess` function are converted to their internal

representation (`OBJECT_ATTRIBUTES` structures, documented in the WDK).

8. `CreateProcessInternalW` checks whether the process should be created as modern. The process is to be created modern if specified so by an attribute (`PROC_THREAD_ATTRIBUTE_PACKAGE_FULL_NAME`) with the full package name or the creator is itself modern (and a parent process has not been explicitly specified by the `PROC_THREAD_ATTRIBUTE_PARENT_PROCESS` attribute). If so, a call is made to the internal `BasepAppXExtension` to gather more contextual information on the modern app parameters described by a structure called `APPX_PROCESS_CONTEXT` . This structure holds information such as the package name (internally referred to as *package moniker*), the capabilities associated with the app, the current directory for the process, and whether the app should have full trust. The option of creating full trust modern apps is not publicly exposed, and is reserved for apps that have the modern look and feel but perform system- level operations. A canonical example is the Settings app in Windows 10 (`SystemSettings.exe`).

9. If the process is to be created as modern, the security capabilities (if provided by `PROC_THREAD_ATTRIBUTE_SECURITY_CAPABILITIES`) are recorded for the initial token creation by calling the internal `BasepCreateLowBox` function. The term *LowBox* refers to the sandbox (`AppContainer`) under which the process is to be executed. Note that although creating modern processes by directly calling `CreateProcess` is not supported (instead, the COM interfaces described earlier should be used), the Windows SDK and MSDN do document the ability to create `AppContainer` legacy desktop applications by passing this attribute.

10. If a modern process is to be created, then a flag is set to indicate to the kernel to skip embedded manifest detection. Modern processes should never have an embedded manifest as it's simply not needed. (A modern app has a manifest of its own, unrelated to the embedded manifest referenced here.)

11. If the debug flag has been specified (`DEBUG_PROCESS`), then the `Debugger` value under the Image File Execution Options registry key (discussed in the next section) for the executable is marked to be skipped. Otherwise, a debugger will never be able to create its debuggee process because the creation will enter an infinite loop (trying to create the debugger process over and over again).

12. All windows are associated with desktops, the graphical representation of a workspace. If no desktop is specified in the `STARTUPINFO` structure, the process is associated with the caller's current desktop.



NOTE

The Windows 10 Virtual Desktop feature does not use multiple desktop objects (in the kernel object sense). There is still one desktop, but windows are shown and hidden as required. This is in contrast to the Sysinternals desktops.exe tool, which really creates up to four desktop objects. The difference can be felt when trying to move a window from one desktop to another. In the case of desktops.exe, it can't be done, as such an operation is not supported in Windows. On the other hand, Windows 10's Virtual Desktop allows it, since there is no real "moving" going on.

13. The application and command-line arguments passed to `CreateProcessInternalW` are analyzed. The executable path name is converted to the internal NT name (for example, `c:\temp\a.exe` turns into something like `\device\harddiskvolume1\temp\a.exe`) because some functions require it in that format.

14. Most of the gathered information is converted to a single large structure of type `RTL_USER_PROCESS_PARAMETERS`.

Once these steps are completed, `CreateProcessInternalW` performs the initial call to `NtCreate-UserProcess` to attempt creation of the process. Because `Kernel32.dll` has no idea at this point whether the ap-

plication image name is a real Windows application or a batch file (.bat or .cmd), 16-bit, or DOS application, the call might fail, at which point `CreateProcessInternalW` looks at the error reason and attempts to correct the situation.

Stage 2: Opening the image to be executed

At this point, the creating thread has switched into kernel mode and continues the work within the `NtCreateUserProcess` system call implementation.

1. `NtCreateUserProcess` first validates arguments and builds an internal structure to hold all creation information. The reason for validating arguments again is to make sure the call to the executive did not originate from a hack that managed to simulate the way `Ntdll.dll` makes the transition to the kernel with bogus or malicious arguments.

2. As illustrated in [Figure 3-11](#), the next stage in `NtCreateUserProcess` is to find the appropriate Windows image that will run the executable file specified by the caller and to create a section object to later map it into the address space of the new process. If the call fails for any reason, it returns to `CreateProcessInternalW` with a failure state (look ahead to [Table 3-8](#)) that causes `CreateProcessInternalW` to attempt execution again.

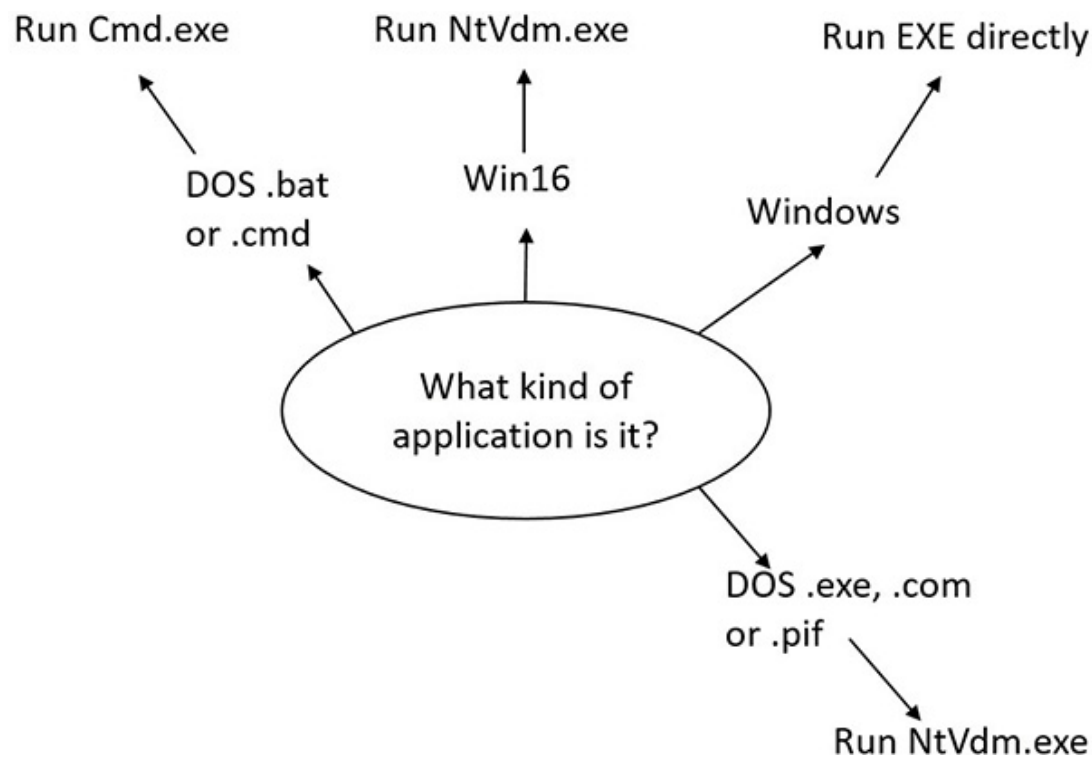


FIGURE 3-11 Choosing a Windows image to activate.

If the Image...	Create State Code	This Image Will Run...	...and This Will Happen
Is an MS-DOS application with an .exe, .com, or .pif extension	PsCreateFailOnSectionCreate	Ntvdm.exe	CreateProcessInternalW restarts stage 1.
Is a Win16 application	PsCreateFailOnSectionCreate	Ntvdm.exe	CreateProcessInternalW restarts stage 1.
Is a Win64 application on a 32-bit system (or a PPC, MIPS, or Alpha binary)	PsCreateFailMachineMismatch	N/A	CreateProcessInternalW will fail.
Has a Debugger value with another image name	PsCreateFailExeName	Name specified in the Debugger value	CreateProcessInternalW restarts stage 1.
Is an invalid or damaged Windows EXE	PsCreateFailExeFormat	N/A	CreateProcessInternalW will fail.
Cannot be opened	PsCreateFailOnFileOpen	N/A	CreateProcessInternalW will fail.
Is a command procedure (application with a .bat or .cmd extension)	PsCreateFailOnSectionCreate	Cmd.exe	CreateProcessInternalW restarts Stage 1.

TABLE 3-8 Decision tree for stage 1 of CreateProcess

3. If the process needs to be created protected, it also checks the signing policy.

4. If the process to be created is modern, a licensing check is done to make sure it's licensed and allowed to run. If the app is inbox (preinstalled with Windows), it's allowed to run regardless of license. If side-

loading apps is allowed (configured through the Settings app), then any signed app can be executed, not just from the store.

5. If the process is a Trustlet, the section object must be created with a special flag that allows the secure kernel to use it.

6. If the executable file specified is a Windows EXE,

`NtCreateUserProcess` tries to open the file and create a section object for it. The object isn't mapped into memory yet, but it is opened. Just because a section object has been successfully created doesn't mean the file is a valid Windows image, however. It could be a DLL or a POSIX executable. If the file is a POSIX executable, the call fails, because POSIX is no longer supported. If the file is a DLL, `CreateProcessInternalW` fails as well.

7. Now that `NtCreateUserProcess` has found a valid Windows executable image, as part of the process creation code described in the next section, it looks in the registry under HKLM\

SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File

Execution Options to see whether a subkey with the file name and extension of the executable image (but without the directory and path information—for example, Notepad.exe) exists there. If it does,

`PspAllocate-Process` looks for a value named `Debugger` for that key. If this value is present, the image to be run becomes the string in that value and `CreateProcessInternalW` restarts at stage 1.



You can take advantage of this process-creation behavior and debug the startup code of Windows services processes before they start rather than attach the debugger after starting a service, which doesn't allow you to debug the startup code.

8. On the other hand, if the image is not a Windows EXE (for example, if it's an MS-DOS or a Win16 application), `CreateProcessInternalW` goes

through a series of steps to find a Windows support image to run it. This process is necessary because non-Windows applications aren't run directly. Windows instead uses one of a few special support images that, in turn, are responsible for actually running the non-Windows program. For example, if you attempt to run an MS-DOS or a Win16 executable (32-bit Windows only), the image to be run becomes the Windows executable Ntvdm.exe. In short, you can't directly create a process that is not a Windows process. If Windows can't find a way to resolve the activated image as a Windows process (as shown in [Table 3-8](#)), `CreateProcessInternalW` fails.

Specifically, the decision tree that `CreateProcessInternalW` goes through to run an image is as follows:

- If it's x86 32-bit Windows, and the image is an MS-DOS application with an .exe, .com, or .pif extension, a message is sent to the Windows subsystem to check whether an MS-DOS support process (Ntvdm.exe, specified in the `HKLM\SYSTEM\CurrentControlSet\Control\WOW\cmdline` registry value) has already been created for this session. If a support process has been created, it is used to run the MS-DOS application. (The Windows subsystem sends the message to the virtual DOS machine [VDM] process to run the new image.) Then `Create-ProcessInternalW` returns. If a support process hasn't been created, the image to be run changes to Ntvdm.exe and `CreateProcessInternalW` restarts at stage 1.
- If the file to run has a .bat or .cmd extension, the image to be run becomes Cmd.exe, the Windows command prompt, and `CreateProcessInternalW` restarts at stage 1. (The name of the batch file is passed as the second parameter to Cmd.exe after the /c switch.)
- For an x86 Windows system, if the image is a Win16 (Windows 3.1) executable, `CreateProcess-InternalW` must decide whether a new VDM process must be created to run it or whether it should use the default session-wide shared VDM process (which might not yet have been created). The `CreateProcess` flags `CREATE_SEPARATE_WOW_VDM` and `CREATE_SHARED_WOW_VDM` control this decision. If these flags aren't spec-

ified, the HKLM\SYSTEM\CurrentControlSet\Control\WOW\DefaultSeparateVDM registry value dictates the default behavior. If the application is to be run in a separate VDM, the image to be run changes to Ntvdm.exe followed by some configuration parameters and the 16-bit process name, and `CreateProcess-InternalW` restarts at stage 1. Otherwise, the Windows subsystem sends a message to see whether the shared VDM process exists and can be used. (If the VDM process is running on a different desktop or isn't running under the same security as the caller, it can't be used, and a new VDM process must be created.) If a shared VDM process can be used, the Windows subsystem sends a message to it to run the new image and `CreateProcessInternalW` returns. If the VDM process hasn't yet been created (or if it exists but can't be used), the image to be run changes to the VDM support image and `CreateProcessInternalW` restarts at stage 1.

Stage 3: Creating the Windows executive process object

At this point, `NtCreateUserProcess` has opened a valid Windows executable file and created a section object to map it into the new process address space. Next, it creates a Windows executive process object to run the image by calling the internal system function `PspAllocateProcess`. Creating the executive process object (which is done by the creating thread) involves the following sub-stages:

3A. Setting up the `EPROCESS` object

3B. Creating the initial process address space

3C. Initializing the kernel process structure (`KPROCESS`)

3D. Concluding the setup of the process address space

3E. Setting up the PEB

3F. Completing the setup of the executive process object



The only time there won't be a parent process is during system initialization (when the System process is created). After that point, a parent process is always required to provide a security context for the new process.

Stage 3A: Setting up the EPROCESS object

This sub-stage involves the following steps:

1. Inherit the affinity of the parent process unless it was explicitly set during process creation (through the attribute list).
2. Choose the ideal NUMA node that was specified in the attribute list, if any.
3. Inherit the I/O and page priority from the parent process. If there is no parent process, the default page priority (5) and I/O priority (Normal) are used.
4. Set the new process exit status to `STATUS_PENDING`.
5. Choose the hard error processing mode selected by the attribute list. Otherwise, inherit the parent's processing mode if none was given. If no parent exists, use the default processing mode, which is to display all errors.
6. Store the parent process's ID in the `InheritedFromUniqueProcessId` field in the new process object.
7. Query the Image File Execution Options (IFEO) key to check if the process should be mapped with large pages (`UseLargePages` value in the IFEO key), unless the process is to run under Wow64, in which case large pages will not be used. Also, query the key to check if NTDLL has been listed as a DLL that should be mapped with large pages within this process.

8. Query the performance options key in IFEO (`PerfOptions` , if it exists), which may consist of any number of the following possible values:

`IoPriority` , `PagePriority` , `CpuPriorityClass` , and `WorkingSetLimitInKB` .

9. If the process would run under Wow64, then allocate the Wow64 auxiliary structure (`EWOW64PROCESS`) and set it in the `Wow64Process` member of the `EPROCESS` structure.

10. If the process is to be created inside an `AppContainer` (in most cases a modern app), validate that the token was created with a LowBox. (See [Chapter 7](#) for more on AppContainers.)

11. Attempt to acquire all the privileges required for creating the process. Choosing the Real-time process priority class, assigning a token to the new process, mapping the process with large pages, and creating the process within a new session are all operations that require the appropriate privilege.

12. Create the process's primary access token (a duplicate of its parent's primary token). New processes inherit the security profile of their parents. If the `CreateProcessAsUser` function is being used to specify a different access token for the new process, the token is then changed appropriately. This change might happen only if the parent token's integrity level dominates the integrity level of the access token, and if the access token is a true child or sibling of the parent token. Note that if the parent has the `SeAssignPrimaryToken` privilege, this will bypass these checks.

13. The session ID of the new process token is now checked to determine if this is a cross-session create. If so, the parent process temporarily attaches to the target session to correctly process quotas and address space creation.

14. Set the new process's quota block to the address of its parent process's quota block, and increment the reference count for the parent's quota block. If the process was created through

`CreateProcessAsUser` , this step won't occur. Instead, the default quota is created, or a quota matching the user's profile is selected.

15. The process minimum and maximum working set sizes are set to the values of `PspMinimumWorkingSet` and `PspMaximumWorkingSet` , respectively. These values can be overridden if performance options were specified in the `PerfOptions` key part of Image File Execution Options, in which case the maximum working set is taken from there. Note that the default working set limits are soft limits and are essentially hints, while the `PerfOptions` working set maximum is a hard limit. (That is, the working set will not be allowed to grow past that number.)

16. Initialize the address space of the process. (See stage 3B.) Then detach from the target session if it was different.

17. The group affinity for the process is now chosen if group-affinity inheritance was not used. The default group affinity will either inherit from the parent if NUMA node propagation was set earlier (the group owning the NUMA node will be used) or be assigned round-robin. If the system is in forced group-awareness mode and group 0 was chosen by the selection algorithm, group 1 is chosen instead, as long as it exists.

18. Initialize the `KPROCESS` part of the process object. (See Stage 3C.)

19. The token for the process is now set.

20. The process's priority class is set to normal unless the parent was using idle or the Below Normal process priority class, in which case the parent's priority is inherited.

21. The process handle table is initialized. If the inherit handles flag is set for the parent process, any inheritable handles are copied from the parent's object handle table into the new process. (For more information about object handle tables, see Chapter 8 in Part 2.) A process attribute can also be used to specify only a subset of handles, which is useful when you are using `CreateProcessAsUser` to restrict which objects should be inherited by the child process.

22. If performance options were specified through the `PerfOptions` key, these are now applied. The `PerfOptions` key includes overrides for the working set limit, I/O priority, page priority, and CPU priority class of the process.

23. The final process priority class and the default quantum for its threads are computed and set.

24. The various mitigation options provided in the IFEO key (as a single 64-bit value named `Mitigation`) are read and set. If the process is under an AppContainer, add the `TreatAs-AppContainer` mitigation flag.

25. All other mitigation flags are now applied.

Stage 3B: Creating the initial process address space

The initial process address space consists of the following pages:

- Page directory (it's possible there'll be more than one for systems with page tables more than two levels, such as x86 systems in PAE mode or 64-bit systems)
- Hyperspace page
- VAD bitmap page
- Working set list

To create these pages, the following steps are taken:

1. Page table entries are created in the appropriate page tables to map the initial pages.
2. The number of pages is deducted from the kernel variable `MmTotalCommittedPages` and added to `MmProcessCommit`.
3. The system-wide default process minimum working set size (`PsmMinimumWorkingSet`) is deducted from `MmResidentAvailablePages`.

4. The page table pages for the global system space (that is, other than the process-specific pages we just described, and except session-specific memory) are created.

Stage 3C: Creating the kernel process structure

The next stage of `PspAllocateProcess` is the initialization of the `KPROCESS` structure (the `Pcb` member of the `EPROCESS`). This work is performed by `KeInitializeProcess`, which does the following:

1. The doubly linked list, which connects all threads part of the process (initially empty), is initialized.
2. The initial value (or reset value) of the process default quantum (which is described in more detail in the “[Thread scheduling](#)” section in [Chapter 4](#)) is hard-coded to 6 until it is initialized later (by `PspComputeQuantumAndPriority`).



NOTE

The default initial quantum differs between Windows client and server systems. For more information on thread quantum, turn to the discussion in the section “[Thread scheduling](#)” in [Chapter 4](#).

-
3. The process’s base priority is set based on what was computed in stage 3A.
 4. The default processor affinity for the threads in the process is set, as is the group affinity. The group affinity was calculated in stage 3A or inherited from the parent.
 5. The process-swapping state is set to resident.
 6. The thread seed is based on the ideal processor that the kernel has chosen for this process (which is based on the previously created process’s ideal processor, effectively randomizing this in a round-robin

manner). Creating a new process will update the seed in `KeNodeBlock` (the initial NUMA node block) so that the next new process will get a different ideal processor seed.

7. If the process is a secure process (Windows 10 and Server 2016), then its secure ID is created now by calling `Hv1CreateSecureProcess`.

Stage 3D: Concluding the setup of the process address space

Setting up the address space for a new process is somewhat complicated, so let's look at what's involved one step at a time. To get the most out of this section, you should have some familiarity with the internals of the Windows memory manager, described in [Chapter 5](#).

The routine that does most of the work in setting the address space is `MmInitializeProcess-AddressSpace`. It also supports cloning an address space from another process. This capability was useful at the time to implement the POSIX `fork` system call. It may also be leveraged in the future to support other Unix-style `fork` (this is how `fork` is implemented in Windows Subsystem for Linux in Redstone 1). The following steps do not describe the address space cloning functionality, but rather focus on normal process address space initialization.

1. The virtual memory manager sets the value of the process's last trim time to the current time. The working set manager (which runs in the context of the balance set manager system thread) uses this value to determine when to initiate working set trimming.
2. The memory manager initializes the process's working set list. Page faults can now be taken.
3. The section (created when the image file was opened) is now mapped into the new process's address space, and the process section base address is set to the base address of the image.
4. The Process Environment Block (PEB) is created and initialized (see the section stage 3E).

5. Ntdll.dll is mapped into the process. If this is a Wow64 process, the 32-bit Ntdll.dll is also mapped.

6. A new session, if requested, is now created for the process. This special step is mostly implemented for the benefit of the Session Manager (Smss) when initializing a new session.

7. The standard handles are duplicated and the new values are written in the process parameters structure.

8. Any memory reservations listed in the attribute list are now processed. Additionally, two flags allow the bulk reservation of the first 1 or 16 MB of the address space. These flags are used internally for mapping, for example, real-mode vectors and ROM code (which must be in the low ranges of virtual address space, where normally the heap or other process structures could be located).

9. The user process parameters are written into the process, copied, and fixed up (that is, they are converted from absolute form to a relative form so that a single memory block is needed).

10. The affinity information is written into the PEB.

11. The *MinWin* API redirection set is mapped into the process and its pointer is stored in the PEB.

12. The process unique ID is now determined and stored. The kernel does not distinguish between unique process and thread IDs and handles. The process and thread IDs (handles) are stored in a global handle table (`PspCidTable`) that is not associated with any process.

13. If the process is secure (that is, it runs in IUM), the secure process is initialized and associated with the kernel process object.

Stage 3E: Setting up the PEB

`NtCreateUserProcess` calls `MmCreatePeb`, which first maps the system-wide National Language Support (NLS) tables into the process's address space. It next calls `MiCreatePebOrTeb` to allocate a page for the PEB and then initializes a number of fields, most of them based on internal variables that were configured through the registry, such as `MmHeap*` values, `MmCriticalSectionTimeout`, and `MmMinimum-StackCommitInBytes`. Some of these fields can be overridden by settings in the linked executable image, such as the Windows version in the PE header or the affinity mask in the load configuration directory of the PE header.

If the image header characteristics `IMAGE_FILE_UP_SYSTEM_ONLY` flag is set (indicating that the image can run only on a uniprocessor system), a single CPU (`MmRotatingUniprocessorNumber`) is chosen for all the threads in this new process to run on. The selection process is performed by simply cycling through the available processors. Each time this type of image is run, the next processor is used. In this way, these types of images are spread evenly across the processors.

Stage 3F: Completing the setup of the executive process object

Before the handle to the new process can be returned, a few final setup steps must be completed, which are performed by `PspInsertProcess` and its helper functions:

1. If system-wide auditing of processes is enabled (because of either local policy settings or group policy settings from a domain controller), the process's creation is written to the Security event log.
2. If the parent process was contained in a job, the job is recovered from the job level set of the parent and then bound to the session of the newly created process. Finally, the new process is added to the job.
3. The new process object is inserted at the end of the Windows list of active processes (`PsActive-ProcessHead`). Now the process is accessi-

ble via functions like `EnumProcesses` and `OpenProcess`.

4. The process debug port of the parent process is copied to the new child process unless the `NoDebugInherit` flag is set (which can be requested when creating the process). If a debug port was specified, it is attached to the new process.

5. Job objects can specify restrictions on which group or groups the threads within the processes part of a job can run on. Therefore, `PspInsertProcess` must make sure the group affinity associated with the process would not violate the group affinity associated with the job. An interesting secondary issue to consider is if the job's permissions grant access to modify the process's affinity permissions, because a lesser-privileged job object might interfere with the affinity requirements of a more privileged process.

6. Finally, `PspInsertProcess` creates a handle for the new process by calling `ObOpenObjectByPointer`, and then returns this handle to the caller. Note that no process-creation callback is sent until the first thread within the process is created, and the code always sends process callbacks before sending object managed-based callbacks.

Stage 4: Creating the initial thread and its stack and context

At this point, the Windows executive process object is completely set up. It still has no thread, however, so it can't do anything yet. It's now time to start that work. Normally, the `PspCreateThread` routine is responsible for all aspects of thread creation and is called by `NtCreateThread` when a new thread is being created. However, because the initial thread is created internally by the kernel without user-mode input, the two helper routines that `PspCreateThread` relies on are used instead: `PspAllocateThread` and `PspInsertThread`. `PspAllocateThread` handles the actual creation and initialization of the executive thread object itself, while `PspInsertThread` handles the creation of the thread handle and security attributes and the call to `KeStartThread` to turn the executive object into a schedulable thread on the system. However, the thread won't do anything yet. It is created in a suspended state and isn't resumed until the process is completely initialized (as described in stage 5).



NOTE

The thread parameter (which can't be specified in `CreateProcess` but can be specified in `CreateThread`) is the address of the PEB. This parameter will be used by the initialization code that runs in the context of this new thread (as described in stage 6).

`PspAllocateThread` performs the following steps:

1. It prevents user-mode scheduling (UMS) threads from being created in Wow64 processes, as well as preventing user-mode callers from creating threads in the system process.
2. An executive thread object is created and initialized.
3. If energy estimation is enabled for the system (always disabled for XBOX), then it allocates and initializes a `THREAD_ENERGY_VALUES` struc-

ture pointed to by the `ETHREAD` object.

4. The various lists used by LPC, I/O Management, and the Executive are initialized.

5. The thread's creation time is set, and its thread ID (TID) is created.

6. Before the thread can execute, it needs a stack and a context in which to run, so these are set up. The stack size for the initial thread is taken from the image; there's no way to specify another size. If this is a Wow64 process, the Wow64 thread context will also be initialized.

7. The thread environment block (TEB) is allocated for the new thread.

8. The user-mode thread start address is stored in the `ETHREAD` (in the `StartAddress` field). This is the system-supplied thread startup function in `Ntdll.dll` (`RtlUserThreadStart`). The user's specified Windows start address is stored in the `ETHREAD` in a different location (the `Win32StartAddress` field) so that debugging tools such as Process Explorer can display the information.

9. `KeInitThread` is called to set up the `KTHREAD` structure. The thread's initial and current base priorities are set to the process's base priority, and its affinity and quantum are set to that of the process.

`KeInitThread` next allocates a kernel stack for the thread and initializes the machine-dependent hardware context for the thread, including the context, trap, and exception frames. The thread's context is set up so that the thread will start in kernel mode in `KiThreadStartup`. Finally, `KeInitThread` sets the thread's state to `Initialized` and returns to `PspAllocateThread`.

10. If this is a UMS thread, `PspUmsInitThread` is called to initialize the UMS state.

Once that work is finished, `NtCreateUserProcess` calls `PspInsertThread` to perform the following steps:

1. The thread ideal processor is initialized if it was specified using an attribute.
2. The thread group affinity is initialized if it was specified using an attribute.
3. If the process is part of a job, a check is made to ensure that the thread's group affinity does not violate job limitations (described earlier).
4. Checks are made to ensure that the process hasn't already been terminated, that the thread hasn't already been terminated, or that the thread hasn't even been able to start running. If any of these are true, thread creation will fail.
5. If the thread is part of a secure process (IUM), then the secure thread object is created and initialized.
6. The `KTHREAD` part of the thread object is initialized by calling `KeStartThread`. This involves inheriting scheduler settings from the owner process, setting the ideal node and processor, updating the group affinity, setting the base and dynamic priorities (by copying from the process), setting the thread quantum, and inserting the thread in the process list maintained by `KPROCESS` (a separate list from the one in `EPROCESS`).
7. If the process is in a deep freeze (meaning no threads are allowed to run, including new threads), then this thread is frozen as well.
8. On non-x86 systems, if the thread is the first in the process (and the process is not the idle process), then the process is inserted into another system-wide list of processes maintained by the global variable `KiProcessListHead`.
9. The thread count in the process object is incremented, and the owner process's I/O priority and page priority are inherited. If this is the highest number of threads the process has ever had, the thread count high

watermark is updated as well. If this was the second thread in the process, the primary token is frozen (that is, it can no longer be changed).

10. The thread is inserted in the process's thread list, and the thread is suspended if the creating process requested it.

11. The thread object is inserted into the process handle table.

12. If it's the first thread created in the process (that is, the operation happened as part of a `Create-Process*` call), any registered callbacks for process creation are called. Then any registered thread callbacks are called. If any callback vetoes the creation, it will fail and return an appropriate status to the caller.

13. If a job list was supplied (using an attribute) and this is the first thread in the process, then the process is assigned to all of the jobs in the job list.

14. The thread is readied for execution by calling `KeReadyThread`. It enters the deferred ready state. (See [Chapter 4](#) for more information on thread states.)

Stage 5: Performing Windows subsystem-specific initialization

Once `NtCreateUserProcess` returns with a success code, the necessary executive process and thread objects have been created.

`CreateProcessInternalW` then performs various operations related to Windows subsystem-specific operations to finish initializing the process.

1. Various checks are made for whether Windows should allow the executable to run. These checks include validating the image version in the header and checking whether Windows application certification has blocked the process (through a group policy). On specialized editions of Windows Server 2012 R2, such as Windows Storage Server 2012 R2, ad-

ditional checks are made to see whether the application imports any disallowed APIs.

2. If software restriction policies dictate, a restricted token is created for the new process. Afterward, the application-compatibility database is queried to see whether an entry exists in either the registry or system application database for the process. Compatibility shims will not be applied at this point; the information will be stored in the PEB once the initial thread starts executing (stage 6).

3. `CreateProcessInternalW` calls some internal functions (for non-protected processes) to get SxS information (see the section “**DLL name resolution and redirection**” later in this chapter for more information on side-by-side) such as manifest files and DLL redirection paths, as well as other information such as whether the media on which the EXE resides is removable and installer detection flags. For immersive processes, it also returns version information and target platform from the package manifest.

4. A message to the Windows subsystem is constructed based on the information collected to be sent to Csrss. The message includes the following information:

- Path name and SxS path name
- Process and thread handles
- Section handle
- The access token handle
- Media information
- AppCompat and shim data
- Immersive process information
- The PEB address

- Various flags such as whether it's a protected process or whether it is required to run elevated
- A flag indicating whether the process belongs to a Windows application (so that Csrss can determine whether to show the startup cursor)
- UI language information
- DLL redirection and `.local` flags (discussed in the “[Image loader](#)” section later in this chapter)
- Manifest file information

When it receives this message, the Windows subsystem performs the following steps:

1. `CsrCreateProcess` duplicates a handle for the process and thread. In this step, the usage count of the process and the thread is incremented from 1 (which was set at creation time) to 2.
2. The Csrss process structure (`CSR_PROCESS`) is allocated.
3. The new process's exception port is set to be the general function port for the Windows subsystem so that the Windows subsystem will receive a message when a second-chance exception occurs in the process. (For further information on exception handling, see Chapter 8 in Part 2.)
4. If a new process group is to be created with the new process serving as the root (`CREATE_NEW_PROCESS_GROUP` flag in `CreateProcess`), then it's set in `CSR_PROCESS` . A process group is useful for sending a control event to a set of processes sharing a console. See the Windows SDK documentation for `CreateProcess` and `GenerateConsoleCtrlEvent` for more information.
5. The Csrss thread structure (`CSR_THREAD`) is allocated and initialized.

6. `CsrCreateThread` inserts the thread in the list of threads for the process.

7. The count of processes in this session is incremented.

8. The process shutdown level is set to `0x280`, the default process shutdown level. (See `SetProcess-ShutdownParameters` in the Windows SDK documentation for more information.)

9. The new Csrss process structure is inserted into the list of Windows subsystem-wide processes.

After Csrss has performed these steps, `CreateProcessInternalW` checks whether the process was run elevated (which means it was executed through `ShellExecute` and elevated by the `AppInfo` service after the consent dialog box was shown to the user). This includes checking whether the process was a setup program. If it was, the process's token is opened, and the virtualization flag is turned on so that the application is virtualized. (See the information on UAC and virtualization in [Chapter 7](#).) If the application contained elevation shims or had a requested elevation level in its manifest, the process is destroyed and an elevation request is sent to the `AppInfo` service.

Note that most of these checks are not performed for protected processes. Because these processes must have been designed for Windows Vista or later, there's no reason they should require elevation, virtualization, or application-compatibility checks and processing.

Additionally, allowing mechanisms such as the shim engine to use its usual hooking and memory-patching techniques on a protected process would result in a security hole if someone could figure how to insert arbitrary shims that modify the behavior of the protected process.

Additionally, because the shim engine is installed by the parent process, which might not have access to its child protected process, even legitimate shimming cannot work.

